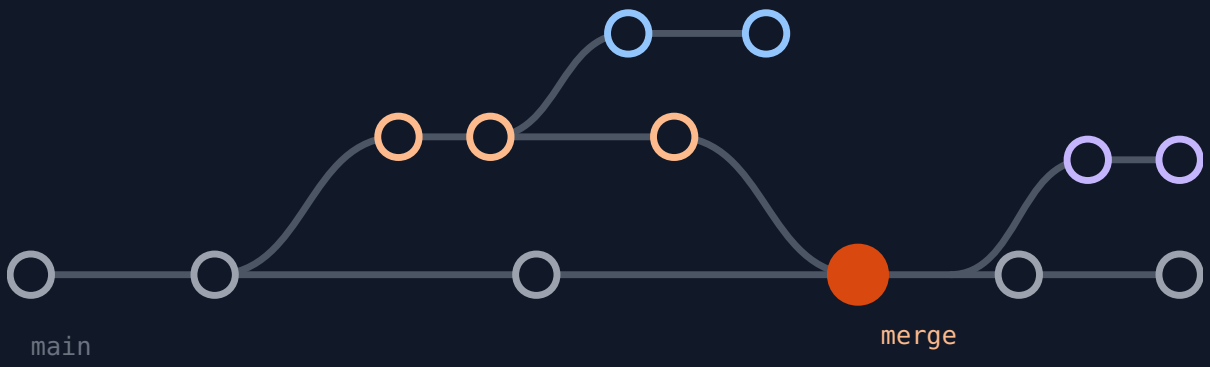
FROM FIRST COMMIT TO PROFESSIONAL PRACTICE

The Complete Git & GitHub Guide

Version control, collaboration, automation and recovery, explained step by step with real commands.



WRITTEN BY

Md Sadman Hafiz

Lecturer, Department of CSE, International Islamic University Chittagong (IIUC)
Founder and Lead, Trustworthy Vision Systems Lab (TVSL)

20 chapters · 4 appendices · 10 hands-on labs

Git basics · Branching & rebasing · GitHub & pull requests · Actions · Security · Research & teaching · Recovery

October 2026 edition

BEFORE YOU BEGIN

Quick Start: The Core Loop

If you read nothing else, learn these commands. They cover most of a normal day with Git and GitHub. Every one is explained in depth later in the guide.

```
# One time: who are you?
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
git config --global init.defaultBranch main

# Start: get a project
git clone git@github.com:user/project.git      # or: git init
cd project

# Daily loop
git switch -c feature/my-change                 # 1. new branch for your work
#    ...edit files...
git status                                      # 2. what changed?
git diff                                        # 3. review the changes
git add -p                                      # 4. stage what belongs together
git commit -m "Add search box to header"        # 5. save a snapshot
git push -u origin feature/my-change            # 6. share it
gh pr create --fill                             # 7. open a pull request

# After the PR is merged
git switch main
git pull                                        # get everyone's latest work
```

Golden rule 1. Run `git status` before and after everything. It tells you where you are and what to do next.

Golden rule 2. Commit small and often. Committed work can almost always be recovered.

Golden rule 3. Never rewrite or force push history that others already have.

Which part should I read?

If you are...	Start with
Completely new to Git	Part I and Part II in order, then Chapters 10 to 12, doing Labs 1 to 7
Using Git but nervous about branches and mistakes	Chapters 3, 6, 8 and 9, then Chapter 20
Joining a team or open source project	Chapters 10 to 12 and 17
Setting up a repository others will use	Chapters 11, 13, 14 and 16
A researcher or teacher	Chapter 19, plus the reproducibility tips in Chapters 14 and 15
In trouble right now	Chapter 20: Troubleshooting and Recovery

Contents

PART I: FOUNDATIONS 7

1 Why Version Control? 8
- The problem every developer meets 8
- Centralized versus distributed 8
- Git and GitHub are different things 9
- A short history 9
- How to use this guide 10

2 Installing and Configuring Git 11
- Installing Git 11
- First-time setup 11
- The three levels of configuration 12
- Aliases: shortcuts for long commands 13
- Getting help 13
- Graphical tools 13

3 How Git Thinks 14
- Snapshots, not differences 14
- The three areas 14
- The lifecycle of a file 14
- What a commit really is 15
- Branches and HEAD are just pointers 15
- Inside the .git folder 16
- The four kinds of objects (optional deep dive) 16
- Everything is local 17

PART II: WORKING LOCALLY 18

4 Your First Repository 19
- Two ways to start 19
- Checking the state: git status 19
- Staging changes: git add 20
- Saving a snapshot: git commit 21
- Writing good commit messages 21
- Ignoring files: .gitignore 22
- Removing and renaming files 23
- Habits of effective committers 23

5 Inspecting Changes and History 25
- Seeing what changed: git diff 25
- Reading history: git log 26
- Naming commits: revision syntax 27
- Looking at one commit: git show 27
- Who wrote this line? git blame 27
- Searching the code: git grep 28
- Statistics: git shortlog 28

6 Undoing Things 29
- The golden question: has it been pushed? 29
- Discarding changes in the working tree 29
- Unstaging a file 29
- Fixing the last commit: --amend 29
- git reset: moving the branch backwards 30

git revert: the safe undo for shared history 30
Restoring an old version of a file 31
Removing untracked files: git clean 31
The reflog: your safety net 31
Quick decision table 32

7 Stashing Work in Progress 33
Why stash? 33
Stash commands 33

8 Branches and Merging 35
Why branches matter 35
Creating, listing and switching 35
Detached HEAD 36
Merging 36
Merge conflicts 37
Cleaning up branches 39

9 Rebasing and Rewriting History 40
What rebase does 40
Conflicts during a rebase 40
Merge or rebase? 41
The golden rule of rebasing 41
Interactive rebase: editing your history 41
Moving a branch to a new base: --onto 42
Pushing after a rebase 43

PART III: REMOTES AND GITHUB 44

10 Working with Remotes 45
What is a remote? 45
Remote-tracking branches 45
fetch, pull and push 46
When a push is rejected 47
Keeping things tidy 47
Several remotes 47

11 Getting Started with GitHub 49
Your account 49
Authenticating Git with GitHub 49
Creating a repository on GitHub 50
The README file 51
Choosing a license 52
Repository settings worth knowing 52
Your profile README 53

12 Collaboration with Pull Requests 54
What is a pull request? 54
The shared-repository workflow 54
Writing a good pull request 55
Reviewing a pull request 55
Updating a pull request 56
Merging: three options 56
The fork workflow (open source) 57

13 Issues, Projects and Communication 59
Issues 59
GitHub Flavored Markdown 60

GitHub Projects 61
Discussions, Wiki and notifications 61
Useful GitHub shortcuts 61

14 GitHub Actions: Automation and CI/CD 63
What GitHub Actions does 63
The vocabulary 63
Your first workflow: run Python tests 63
Common triggers 65
Secrets and variables 65
Saving build outputs: artifacts 65
A status badge 66
Security and good practice 66
Debugging failed workflows 66

15 Tags, Releases, Pages and More 67
Tags: naming important commits 67
GitHub Releases 67
GitHub Pages: free websites 68
Other GitHub features 68

16 Security and Repository Protection 69
Protecting important branches 69
CODEOWNERS 69
Dependabot 70
Secret scanning and push protection 70
Code scanning 70
Signing commits 70
Security checklist 71

PART IV: PROFESSIONAL PRACTICE 72

17 Team Workflows and Conventions 73
GitHub Flow 73
Git Flow 73
Trunk-based development 74
Forking workflow 74
Which workflow should you choose? 74
Conventional Commits 74
A day in the life of a team developer 75
Team agreements worth writing down 75

18 Advanced Git Tools 76
Cherry-pick: copy a single commit 76
Bisect: find the commit that broke things 76
Worktrees: several branches checked out at once 76
Submodules: a repository inside a repository 77
Git LFS: large files 77
Hooks: run scripts automatically 78
.gitattributes: per-file behaviour 78
Big repositories: shallow clones and sparse checkout 79
rerere: remember conflict resolutions 79
Ignoring formatting commits in blame 79
Rewriting a whole history: git filter-repo 79
Other handy commands 80

19 Git for Research and Teaching 81
A clean structure for a research repository 81

Making results reproducible 81
Jupyter notebooks and Git 82
Data: what to do with it 82
Writing papers with Git 82
Making research software citable 83
Git in the classroom 83

20 Troubleshooting and Recovery 85
"I committed to the wrong branch" (not pushed yet) 85
"I need to undo a merge or pull" 85
"Push rejected: fetch first / non-fast-forward" 85
"fatal: refusing to merge unrelated histories" 86
"Permission denied (publickey)" 86
"Authentication failed" or "Support for password authentication was removed" 86
"Large files detected / file exceeds GitHub's 100 MB limit" 86
"I committed a password, key or token" 87
"I am in detached HEAD and made commits" 87
"I deleted a branch by mistake" 87
"I ran git reset --hard and lost commits" 88
"I dropped a stash" 88
"Wrong name or email on my commits" 88
"Remove one file from the last commit" 88
"Every file shows as modified, but I changed nothing" 88
"My fork is many commits behind" 89
"Git asks me to write a message in a strange editor" 89
"A merge or rebase went wrong and I am lost" 89
When nothing else helps 89

Appendix A: Command Cheat Sheet 90

Appendix B: Glossary 93

Appendix C: Hands-on Exercises 95

Appendix D: Further Resources 97

PART I

Foundations

What version control is, how to set Git up properly, and the mental model that makes every later command easy to understand.

CHAPTER 1

Why Version Control?

1.1 The problem every developer meets

Most people start managing their work like this:

```
report.docx
report_v2.docx
report_v2_final.docx
report_v2_final_REALLY_final.docx
report_v3_teacher_comments.docx
```

This works for a while. Then it breaks. You cannot tell what changed between two copies. You cannot tell who changed it. Two people edit the same file and one person's work is lost. A bug appears and you have no idea which edit caused it.

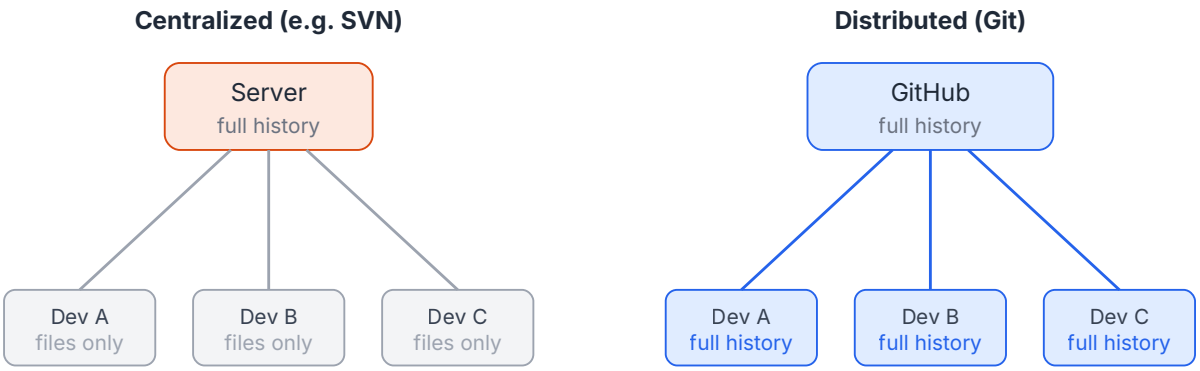
A **version control system (VCS)** solves these problems. It records every change to a set of files over time, so that you can:

- **See history.** Who changed what, when, and why.
- **Go back.** Restore any file, or the whole project, to any earlier state.
- **Work in parallel.** Try an idea on a separate line of work without breaking the main version.
- **Collaborate.** Many people can change the same project and combine their work safely.
- **Back up.** Every copy of the project is a full backup of its entire history.
- **Find bugs.** Search history to find the exact change that introduced a problem.

1.2 Centralized versus distributed

Older systems such as Subversion (SVN) are **centralized**. There is one server with the history. Developers check out files from it and send changes back to it. If the server is down, nobody can commit. If the server is lost, the history is lost.

Git is **distributed**. When you copy (clone) a project, you get the *complete* history on your own computer. You can commit, branch, view history and undo changes with no network at all. You only need the network to share work with others.



In a distributed system every developer has the full history, not just the latest files.

1.3 Git and GitHub are different things

This confuses many beginners, so it is worth being precise.

	Git	GitHub
What it is	A version control **program**	A **website and service** that hosts Git repositories
Where it runs	On your computer	In the cloud (github.com)
Made by	Linus Torvalds and the Git community (2005)	GitHub, Inc., now owned by Microsoft
Cost	Free and open source	Free plan plus paid plans
Main job	Track changes, branch, merge	Share code, review it, manage issues, run automation
Works without the other?	Yes, fully	No, it is built on Git

Think of Git as the engine and GitHub as one of several garages where you can park and share the car. Other hosting services include **GitLab**, **Bitbucket**, **Codeberg** and self-hosted **Gitea** or **Forgejo**. Almost everything you learn about Git in Parts I and II works the same on all of them.

1.4 A short history

Git was created by Linus Torvalds in 2005 to manage the Linux kernel, after the project lost free access to its previous tool, BitKeeper. His goals were speed, a simple design, strong support for thousands of parallel branches, and the ability to handle very large projects. Those goals explain much of how Git behaves today. GitHub launched in 2008 and made Git the default choice for open source and, later, for most companies.

1.5 How to use this guide

The guide moves from basic to advanced. Parts I and II teach Git on your own computer. Part III teaches GitHub. Part IV covers team workflows, advanced tools, research use and recovery from mistakes. The appendices give a cheat sheet, a glossary and practice exercises.

Conventions used in this guide:

- Commands you type are shown in code boxes. Lines starting with `#` are comments that explain the command; you do not type them.
- Text in angle brackets is a placeholder. For `git switch <branch>`, type something like `git switch login-page`.
- Output from Git is shown in separate boxes labelled as output, sometimes shortened with `...`.
- Commit IDs in examples (like `a1b2c3d`) will be different on your computer.

> **Learn by doing.** Open a terminal next to this guide and type the commands yourself. Create a practice folder that you do not mind breaking. Git is best learned with your hands, not only your eyes.

CHAPTER 2

Installing and Configuring Git

2.1 Installing Git

Windows. Download *Git for Windows* from `git-scm.com` and run the installer. It includes **Git Bash**, a terminal that understands Linux-style commands; this guide assumes you use it. The default options are fine for most people. Pay attention to three screens:

1. *Default editor.* Choose an editor you know, such as VS Code or Notepad++, instead of Vim if Vim is new to you.
2. *Initial branch name.* Choose "Override the default branch name" and type `main`.
3. *Line endings.* Keep "Checkout Windows-style, commit Unix-style line endings".

You can also install from the command line with `winget install --id Git.Git -e`.

macOS. Run `git --version` in Terminal. If Git is not installed, macOS offers to install the Command Line Developer Tools, which include Git. For a newer version, use Homebrew: `brew install git`.

Linux. Use your package manager:

```
sudo apt install git        # Debian, Ubuntu
sudo dnf install git        # Fedora
sudo pacman -S git          # Arch
```

Check that it works:

```
git --version
```

```
OUTPUT
git version 2.43.0
```

Any recent 2.x version is fine for this guide. Commands such as `git switch` and `git restore` need Git 2.23 or newer.

2.2 First-time setup

Git stamps your name and email on every commit you make. Set them once:

```
git config --global user.name  "Your Name"
git config --global user.email "you@example.com"
```

> **Which email?** Use the same email as your GitHub account so GitHub links commits to your profile. If you do not want your real email to be public, GitHub gives you a private "noreply" address (Settings, then Emails). Use that address here instead.

Next, set some sensible defaults:

```
# Name the first branch "main" in new repositories
git config --global init.defaultBranch main

# Choose your editor for commit messages
git config --global core.editor "code --wait"     # VS Code
# git config --global core.editor "nano"          # simple terminal editor

# Colour output (usually on by default)
git config --global color.ui auto

# How "git pull" combines work: merge (false) or rebase (true)
git config --global pull.rebase false
```

Line endings. Windows ends lines with two characters (CRLF); macOS and Linux use one (LF). Mixed line endings create noisy diffs. A safe setting:

```
# On Windows
git config --global core.autocrlf true
# On macOS and Linux
git config --global core.autocrlf input
```

For a team project, a `.gitattributes` file in the repository is even better, because it applies to everyone (see the chapter on advanced tools).

2.3 The three levels of configuration

Git reads settings from three places. A more specific level overrides a more general one.

Level	Flag	File	Applies to
System	`--system`	`/etc/gitconfig`	Every user on the computer
Global	`--global`	`~/.gitconfig`	Every repository of the current user
Local	`--local` (default)	`.git/config`	One repository only

A common use of local settings: use your university email in a work repository and your personal email in a hobby repository.

```
cd my-work-project
git config user.email "you@university.edu"   # local, this repo only
```

To see every setting and where it comes from:

```
git config --list --show-origin
git config user.email          # show one value
```

2.4 Aliases: shortcuts for long commands

```
git config --global alias.st  "status -sb"
git config --global alias.lg  "log --oneline --graph --decorate --all"
git config --global alias.last "log -1 HEAD --stat"
git config --global alias.unstage "restore --staged"
```

Now `git lg` prints a compact graph of all branches. Aliases save a lot of typing over time.

2.5 Getting help

```
git help commit        # full manual page
git commit -h          # short summary of options
git help -g            # list of concept guides
```

The free book *Pro Git* at `git-scm.com/book` is the best long reference.

2.6 Graphical tools

You do not have to use only the terminal. Popular options:

- **VS Code** has a built-in *Source Control* panel and excellent merge-conflict tools.
- **GitHub Desktop** is simple and friendly for beginners.
- **GitKraken**, **Sourcetree** and **Fork** draw clear branch graphs.
- **lazygit** and **tig** are fast text interfaces for the terminal.

> **Learn the command line first.** GUIs hide what is happening. When something goes wrong, you need to understand the commands behind the buttons. Once you understand them, use whichever tool makes you fastest.

CHAPTER 3

How Git Thinks

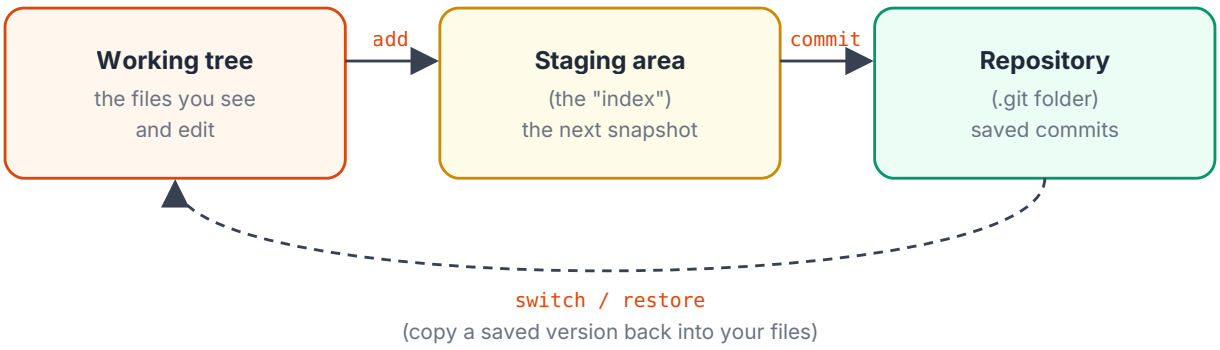
Most Git confusion comes from a wrong mental model. Spend time on this chapter. Every later command will make more sense.

3.1 Snapshots, not differences

Many older tools store a project as a base file plus a list of changes. Git instead stores **snapshots**. Each time you commit, Git takes a picture of what *all* your tracked files look like at that moment and stores a reference to that snapshot. If a file has not changed, Git does not store it again; it just points to the copy it already has. This design makes branching, switching and comparing very fast.

3.2 The three areas

This is the single most important idea in Git. Your work lives in three places:



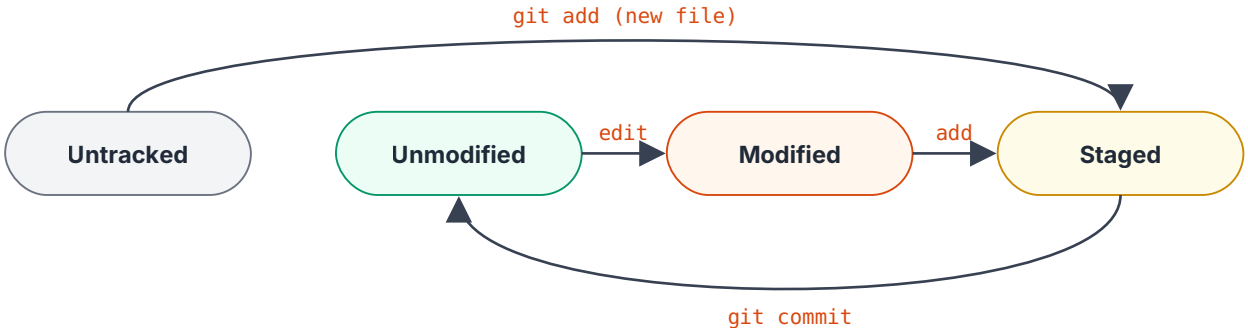
The three areas of a Git project and the commands that move work between them.

1. **Working tree (working directory).** The normal files and folders on your disk. You edit them with any editor. Git watches them but does not save anything automatically.
2. **Staging area (index).** A preparation zone. You choose exactly which changes go into the next commit by adding them here with `git add`.
3. **Repository.** The `.git` folder. It holds every commit ever made. `git commit` takes what is in the staging area and saves it here permanently.

Why a staging area? It lets you make *clean, focused commits*. Imagine you fixed a bug and also reformatted an unrelated file. You can stage and commit the bug fix alone with a clear message, then commit the formatting separately. Each commit then tells one clear story.

3.3 The lifecycle of a file

Every file in your project is in one of these states:



A file moves between four states as you edit, stage and commit it.

- **Untracked:** Git sees the file but has never saved it. New files start here.
- **Unmodified:** Tracked, and identical to the last commit.
- **Modified:** Tracked, and changed since the last commit, but not staged.
- **Staged:** The change is marked to go into the next commit.

`git status` always tells you which state each file is in.

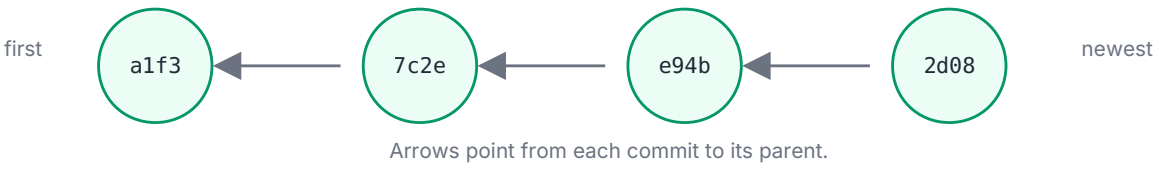
3.4 What a commit really is

A **commit** is a saved snapshot plus some information about it:

- a pointer to the full snapshot of the project,
- the author's name, email and time,
- the committer's name, email and time (usually the same person),
- a message explaining the change,
- a pointer to the **parent** commit (the one before it). A merge commit has two or more parents.

Each commit is identified by a **hash**: a 40-character code calculated from its content, such as `9fceb02d0ae598e95dc970b74767f19372d61af8`. You usually only need the first 7 or so characters (`9fceb02`). Because the hash comes from the content, changing anything in a commit (even one letter of the message) produces a different hash. This makes Git history very hard to corrupt silently.

Because each commit points to its parent, commits form a chain:

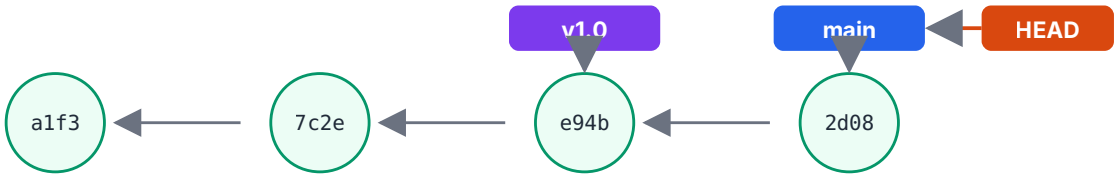


History is a chain of commits, each pointing back to its parent.

3.5 Branches and HEAD are just pointers

A **branch** in Git is not a copy of your files. It is a tiny file containing the hash of one commit. When you make a new commit on a branch, the branch pointer moves forward to the new commit automatically.

HEAD is a special pointer that means "where you are now". Normally HEAD points to a branch, and that branch points to a commit.



HEAD points to the branch main, which points to the latest commit. The tag v1.0 permanently marks an older commit.

Because a branch is just a 41-byte file, creating one is instant, even in a huge project. This is why Git users create branches freely, often several per day.

3.6 Inside the .git folder

When you run `git init`, Git creates a hidden `.git` folder. You never need to edit it by hand, but knowing what is inside removes the mystery:

Item	Purpose
`HEAD`	Says which branch you are on, e.g. `ref: refs/heads/main`
`config`	Settings for this repository (local level)
`objects/`	The database: every file version, folder listing and commit
`refs/heads/`	One file per local branch, each containing a commit hash
`refs/tags/`	One file per tag
`refs/remotes/`	Your last known positions of branches on remote servers
`index`	The staging area
`hooks/`	Scripts that can run automatically at certain events
`logs/`	The reflog: a diary of where HEAD and branches have been

> **Never delete the .git folder** unless you really want to delete all history. If you delete it, your files remain, but the project is no longer a Git repository and every commit is gone (unless a copy exists somewhere else, such as GitHub).

3.7 The four kinds of objects (optional deep dive)

Git's database stores four kinds of objects. You can inspect them with `git cat-file`.

- **Blob:** the content of one file (no name, just the bytes).
- **Tree:** a folder listing that maps names to blobs and other trees.
- **Commit:** points to one tree (the project snapshot) plus parents, author and message.
- **Tag:** an annotated tag with its own message and author, pointing to a commit.

```
git cat-file -p HEAD            # show the latest commit object
```

```
OUTPUT
tree 4b825dc642cb6eb9a060e54bf8d69288fbee4904
parent 7c2e1d0b8a...
author Ayesha Rahman <ayesha@example.com> 1761400000 +0600
committer Ayesha Rahman <ayesha@example.com> 1761400000 +0600

Add login form validation
```

```
git cat-file -p HEAD^{tree}     # show the folder listing of that snapshot
```

```
OUTPUT
100644 blob e69de29bb2...    README.md
040000 tree 9a8b7c6d5e...    src
```

You will rarely need these commands day to day, but they show that Git is a simple, elegant content database underneath.

3.8 Everything is local

Almost every Git command works only on your own computer: `commit`, `log`, `diff`, `branch`, `merge`, `switch`. Only four common commands talk to a remote server: `clone`, `fetch`, `pull` and `push`. This is why Git feels fast, and why you can work on a train or during a power cut and share later.

Chapter summary

- Git stores snapshots of your whole project, not lists of differences.
- Work moves from the working tree to the staging area (`git add`) to the repository (`git commit`).
- A commit is a snapshot plus author, message and parent, identified by a hash.
- A branch is a movable pointer to a commit. HEAD shows where you are.
- Nearly everything happens locally; only clone, fetch, pull and push use the network.

PART II

Working Locally

The everyday commands: saving work, reading history, undoing mistakes, stashing, branching, merging and rebasing. Everything in this part works without the internet.

CHAPTER 4

Your First Repository

4.1 Two ways to start

There are only two ways to get a Git repository:

1. **Turn an existing folder into a repository** with `git init`.
2. **Copy an existing repository** from somewhere else with `git clone`.

Starting from scratch with git init

```
mkdir my-project
cd my-project
git init
```

```
OUTPUT
Initialized empty Git repository in /home/ayesha/my-project/.git/
```

That is all. The folder is now a repository. Nothing is tracked yet; Git is only watching.

You can also run `git init` inside a folder that already has files, such as an old project. Your files stay exactly as they are.

Copying a project with git clone

```
git clone https://github.com/octocat/Hello-World.git
cd Hello-World
```

Cloning downloads the whole history, sets up a connection to the original called `origin`, and checks out the default branch. To clone into a folder with a different name:

```
git clone https://github.com/octocat/Hello-World.git hello
```

> **Do not create a repository inside another repository** by accident. Check with `git status` before running `git init`. If it says "On branch ...", you are already inside a repository. Nested repositories cause confusing behaviour (the proper way to nest is a *submodule*, covered later).

4.2 Checking the state: git status

`git status` is the command you will run most often. Run it before and after almost everything.

```
echo "# My Project" > README.md
echo "print('hello')" > app.py
git status
```

```
OUTPUT
On branch main

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
    README.md
    app.py

nothing added to commit but untracked files present (use "git add" to track)
```

Git reads like a helpful assistant: it tells you where you are and suggests the next command. For a compact view, use the short format:

```
git status -s
```

```
OUTPUT
A  README.md
?? app.py
 M style.css
MM index.html
```

The two columns show the staging area (left) and the working tree (right). `??` means untracked, `A` added, `M` modified, `D` deleted, `R` renamed. `MM` means the file was staged, then changed again after staging.

4.3 Staging changes: git add

```
git add README.md          # stage one file
git add src/               # stage a whole folder
git add *.py               # stage by pattern
git add .                  # stage everything in the current folder and below
git add -A                 # stage every change in the whole repository
```

Staging part of a file

Sometimes one file contains two unrelated changes. Patch mode lets you choose piece by piece:

```
git add -p app.py
```

Git shows each change (a "hunk") and asks what to do. The most useful answers: `y` stage this hunk, `n` skip it, `s` split it into smaller hunks, `e` edit it by hand, `q` quit, `?` help. Patch mode is the secret to clean commits. Good developers use it daily.

4.4 Saving a snapshot: git commit

```
git commit -m "Add README and hello script"
```

```
OUTPUT
[main (root-commit) 9fcfb66] Add README and hello script
 2 files changed, 2 insertions(+)
 create mode 100644 README.md
 create mode 100644 app.py
```

Without `-m`, Git opens your editor so you can write a longer message. Save and close the editor to finish the commit. An empty message cancels the commit.

Shortcut. `git commit -a -m "..."` stages all *modified tracked* files and commits in one step.

> `git commit -a` does **not** include new (untracked) files. You still need `git add` for those. Many beginners forget this and wonder why a new file is missing from GitHub.

4.5 Writing good commit messages

Your commit history is documentation. In six months you, your teammates and your examiners will read it. A good message explains **what** changed and, more importantly, **why**.

The common format:

```
Short summary in the imperative mood (50 chars or less)

More detailed explanation, wrapped at about 72 characters. Explain the
problem this commit solves and why you chose this approach. The code
already shows *how*; the message should explain *why*.

- Bullet points are fine
- Mention related issues: Fixes #42
```

Rules of thumb:

1. Write the summary as a command: "Add", "Fix", "Remove", "Refactor". Read it as "If applied, this commit will *add login validation*."
2. Keep the summary short (around 50 characters). GitHub cuts off long lines.
3. Leave a blank line between the summary and the body.
4. One logical change per commit. If your message needs the word "and" twice, split the commit.

Weak message	Better message
`update`	`Fix crash when uploading empty CSV file`
`changes`	`Add email validation to registration form`
`fixed bug`	`Prevent duplicate orders on double click`
`final version!!`	`Prepare release 1.2.0`

Weak message	Better message
`stuff`	`Move database settings to environment variables`

Some teams use a stricter format called *Conventional Commits* (`feat:`, `fix:`, `docs:`). See the Team Workflows chapter.

4.6 Ignoring files: .gitignore

Some files should never be committed: compiled output, dependency folders, logs, editor settings and, above all, **secrets** such as passwords and API keys. List them in a file named `.gitignore` at the top of the repository.

.GITIGNORE

```
# Python
__pycache__/
*.pyc
.venv/
.ipynb_checkpoints/

# Node.js
node_modules/
dist/

# Secrets and local settings
.env
*.pem
config.local.json

# Operating system and editor junk
.DS_Store
Thumbs.db
.vscode/
.idea/

# LaTeX build files
*.aux
*.log
*.synctex.gz

# Large data and model files
data/raw/
*.ckpt
*.pt
```

Pattern rules:

Pattern	Meaning
`file.txt`	Any file named `file.txt` in any folder
`/file.txt`	Only `file.txt` at the repository root
`build/`	Any folder named `build` and everything in it
`*.log`	Any file ending in `.log`

Pattern	Meaning
`logs/**/*.tmp`	`.tmp` files at any depth inside `logs`
`!keep.log`	Exception: do track `keep.log` even though `*.log` is ignored
`#`	Comment line

Commit the `.gitignore` file itself so the whole team shares it. GitHub keeps good templates for every language at `github.com/github/gitignore`, and offers them when you create a repository.

"I added it to .gitignore but Git still tracks it"

`.gitignore` only affects **untracked** files. If a file was committed before, you must stop tracking it while keeping it on disk:

```
git rm --cached .env           # stop tracking, keep the local file
git commit -m "Stop tracking .env"
```

To find out why a file is ignored:

```
git check-ignore -v path/to/file
```

A personal global ignore file is useful for files that only you create, such as your editor's settings:

```
git config --global core.excludesFile ~/.gitignore_global
```

> **Never commit secrets.** If you commit a password or API key and push it, assume it is stolen, even if you delete it a minute later. Automated bots scan GitHub for keys within seconds. Change (rotate) the secret immediately. See the Troubleshooting chapter for how to remove it from history.

4.7 Removing and renaming files

```
git rm old_file.py             # delete the file and stage the deletion
git rm --cached big_data.csv   # stop tracking but keep the file on disk
git mv utils.py helpers.py     # rename and stage the rename
```

If you delete or rename a file with your normal file manager instead, that is fine. `git add -A` will notice and stage the change. Git detects renames automatically by comparing content.

4.8 Habits of effective committers

- **Commit early and often.** Small commits are easy to understand, review and undo.
- **Each commit should leave the project working.** Avoid committing half-written code to a shared branch.
- **Review before committing.** Run `git status` and `git diff --staged` to check exactly what you are about to save.
- **Do not commit generated files.** If a tool can rebuild it, ignore it.

Try it. Create a folder `practice`, run `git init`, create three files, and make three separate commits, one per file, each with a good message. Then add `notes.tmp`, add `*.tmp` to `.gitignore`, and confirm with `git status` that `notes.tmp` is no longer listed.

CHAPTER 5

Inspecting Changes and History

5.1 Seeing what changed: git diff

`git diff` compares two versions and shows the lines that differ. Which two versions depends on the arguments:

Command	Compares
`git diff`	Working tree vs staging area (unstaged changes)
`git diff --staged`	Staging area vs last commit (what will be committed)
`git diff HEAD`	Working tree vs last commit (all changes)
`git diff main feature`	Tip of `main` vs tip of `feature`
`git diff main...feature`	Changes on `feature` since it split from `main`
`git diff a1b2c3d e4f5g6h`	Any two commits
`git diff HEAD~3 -- app.py`	One file, compared with three commits ago
`git diff --stat`	Summary: which files changed and how much
`git diff --word-diff`	Shows changed words, useful for prose and LaTeX

Reading a diff

OUTPUT

```
diff --git a/app.py b/app.py
index b376c99..5acade7 100644
--- a/app.py
+++ b/app.py
@@ -1 +1,2 @@
 print('hello')
+print('bye')
```

- `--- a/` is the old version and `+++ b/` the new version.
- `@@ -1 +1,2 @@` is the *hunk header*: starting at line 1, the old file had 1 line here; the new file has 2 lines.
- Lines starting with a space are unchanged context, `+` lines were added, `-` lines were removed. A changed line appears as one `-` and one `+`.

5.2 Reading history: git log

```
git log
```

```
OUTPUT
commit 2d08a7e4c1b9f3e6a5d7c8b9a0e1f2d3c4b5a6e7 (HEAD -> main, origin/main)
Author: Ayesha Rahman <ayesha@example.com>
Date:   Sat Oct 24 14:02:11 2026 +0600

    Add login form validation

commit e94b31c...
```

Press `q` to quit the log viewer, the space bar for the next page, and `/` to search.

The plain log is long. These options make it far more useful:

```
git log --oneline                       # one line per commit
git log --oneline --graph --all         # draw all branches as a graph
git log -5                              # only the last 5 commits
git log --stat                          # files changed in each commit
git log -p                              # full diff of each commit
git log --author="Ayesha"               # commits by one person
git log --since="2 weeks ago"           # recent commits
git log --since=2026-09-01 --until=2026-09-30
git log --grep="login"                  # commits whose message mentions "login"
git log -S "calculate_tax"              # commits that added or removed this text
git log -- src/payment.py               # commits that touched one file
git log --follow -- helpers.py          # follow the file through renames
git log main..feature                   # commits on feature that are not on main
```

`git log -S` (the "pickaxe") is a superpower. It finds the commit where a function name, a setting or a magic number first appeared or disappeared.

A graph view looks like this:

```
OUTPUT
* 7f3c2a1 (HEAD -> main) Merge branch 'login-page'
|\
| * 4b8e9d0 (login-page) Add password strength meter
| * 1c7a5f2 Add login form
* | 9e2d6b3 Fix typo in README
|/
* 3a9f1e8 Initial commit
```

A beautiful custom log

```
git log --graph --pretty=format:'%C(yellow)%h%Creset %C(cyan)%an%Creset %s %C(green)(%cr)
%Creset%C(auto)%d'
```

Save it as an alias (Chapter 2) and you will never type it again.

5.3 Naming commits: revision syntax

Many commands accept a commit. You can name one in several ways:

Syntax	Meaning
`a1b2c3d`	A commit by (short) hash
`HEAD`	The current commit
`HEAD~1` or `HEAD~`	The parent of HEAD (one commit back)
`HEAD~3`	Three commits back, following first parents
`HEAD^2`	The *second* parent of a merge commit
`main`	The commit the branch `main` points to
`origin/main`	Your last known position of `main` on the remote
`v1.0`	The commit a tag points to
`main@{yesterday}`	Where `main` was yesterday (from the reflog)
`HEAD@{2}`	Where HEAD was two moves ago (from the reflog)
`main..feature`	Commits in `feature` but not in `main`
`main...feature`	Commits in either branch but not in both

5.4 Looking at one commit: git show

```
git show a1b2c3d                # message and full diff of a commit
git show HEAD~2 --stat          # files changed two commits ago
git show v1.0:src/config.py     # a file as it was at tag v1.0
```

The last form is handy: you can view (or save) any old version of a file without changing anything.

```
git show HEAD~5:report.tex > old_report.tex
```

5.5 Who wrote this line? git blame

```
git blame app.py
git blame -L 40,60 app.py       # only lines 40 to 60
git blame -w app.py             # ignore whitespace-only changes
```

Each line is printed with the commit, author and date that last changed it. Despite the name, use it to find *context*, not to blame people. Once you have the commit hash, `git show <hash>` tells you why the line was written.

> On GitHub, open any file and click **Blame** to get the same view in your browser, with links to each commit and pull request.

5.6 Searching the code: git grep

`git grep` searches tracked files only, so it skips `node_modules` and other ignored folders. It is fast and can search old versions too.

```
git grep "TODO"
git grep -n "def train" -- "*.py"   # with line numbers, only Python files
git grep "API_URL" v1.0             # search the code as it was at v1.0
```

5.7 Statistics: git shortlog

```
git shortlog -sn                    # number of commits per author
git shortlog -sn --since="1 month ago"
```

This is useful for teachers who want to see each student's contribution to a group project, and for open source maintainers preparing release notes.

Chapter summary

- `git diff` shows unstaged changes; `git diff --staged` shows what you are about to commit.
- `git log --oneline --graph --all` gives the best overview of history.
- `git log -S "text"` finds when a piece of code appeared or disappeared.
- `HEAD~n`, `branch..branch` and `file@commit` syntax let you point at any version.
- `git show commit:path` displays an old version of a file without changing anything.

CHAPTER 6

Undoing Things

Mistakes are normal. Git is designed so that almost nothing is truly lost, as long as it was committed at least once. This chapter is your safety manual.

6.1 The golden question: has it been pushed?

Before undoing anything, ask: **Have I already shared this commit with others (pushed it)?**

- **Not pushed (only on my computer):** You may freely rewrite history. Use `amend`, `reset` and interactive rebase.
- **Already pushed and others may have it:** Do not rewrite. Use `git revert` to create a new commit that undoes the old one.

Rewriting shared history forces everyone else to repair their copies. On a team, this causes lost work and angry messages.

6.2 Discarding changes in the working tree

You edited a file and want to throw the edits away, returning to the last committed version:

```
git restore app.py          # discard unstaged changes in one file
git restore .               # discard all unstaged changes
```

`git restore <file>` permanently deletes your uncommitted edits. Git never saved them, so it cannot bring them back. If in doubt, run `git stash` instead, which saves the changes on a shelf.

Older tutorials use `git checkout -- app.py` for this. It still works, but `git restore` is clearer.

6.3 Unstaging a file

You ran `git add` on something by mistake. Remove it from the staging area but keep your edits:

```
git restore --staged app.py
```

The older equivalent is `git reset HEAD app.py`.

6.4 Fixing the last commit: --amend

Typo in the message? Forgot to add a file? If you have **not pushed yet**, amend the commit:

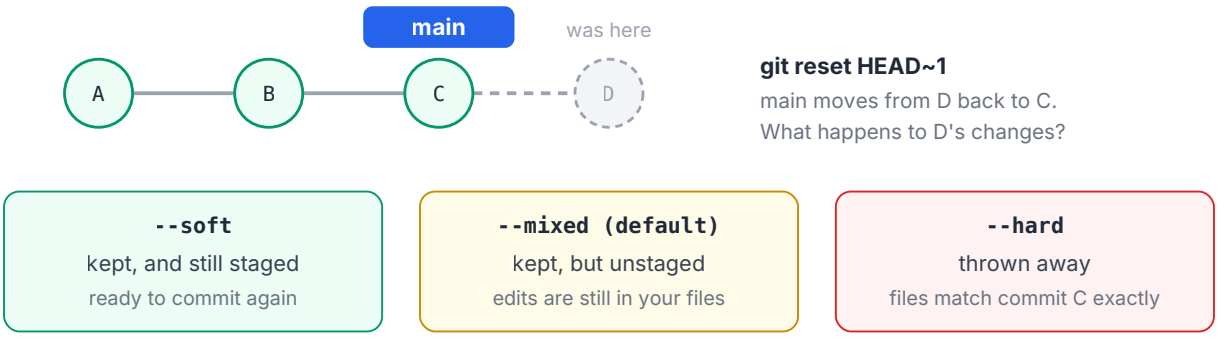
```
# Change only the message
git commit --amend -m "Add login form validation"

# Add a forgotten file to the last commit, keep the message
git add forgotten_file.py
git commit --amend --no-edit
```

Amending replaces the last commit with a new one (it gets a new hash). That is why you must not amend a commit you have already pushed to a shared branch.

6.5 git reset: moving the branch backwards

`git reset <commit>` moves the current branch pointer back to an older commit. The three modes decide what happens to the changes from the commits you "removed":



The three modes of git reset differ only in what they do with the undone changes.

```
git reset --soft HEAD~1    # undo the commit, keep changes staged
git reset HEAD~1           # undo the commit, keep changes as unstaged edits
git reset --hard HEAD~1    # undo the commit AND delete its changes
```

Common uses:

- *"I committed too early."* Use `--soft HEAD~1`, fix things, commit again.
- *"Squash my last three messy commits into one."* Use `git reset --soft HEAD~3`, then `git commit -m "One clean message"`.
- *"My experiment failed; go back to the last commit."* Use `git reset --hard HEAD` (this also discards uncommitted edits).
- *"Make my branch exactly match the server."* Use `git fetch` then `git reset --hard origin/main`.

> `--hard` deletes uncommitted work permanently. Committed work "removed" by reset can usually be recovered with the reflog (below) for a few weeks, but uncommitted edits cannot.

6.6 git revert: the safe undo for shared history

`git revert` does not delete anything. It creates a **new** commit that applies the opposite of an old commit. History stays intact, so it is safe on shared branches.

```
git revert a1b2c3d              # undo one commit with a new commit
git revert HEAD~3..HEAD         # undo the last three commits (one revert each)
git revert --no-commit HEAD~3..HEAD   # undo them, then commit once yourself
```

Reverting a merge commit needs one extra flag to say which parent is the "main line" (usually 1):

```
git revert -m 1 <merge-commit-hash>
```

6.7 Restoring an old version of a file

To bring one file back to how it was in an older commit, without touching anything else:

```
git restore --source=HEAD~3 app.py
git restore --source=v1.0 -- docs/
```

The restored version appears as a normal change you can review and commit.

6.8 Removing untracked files: git clean

Build output and temporary files pile up. `git clean` deletes untracked files. Always do a dry run first:

```
git clean -n         # show what WOULD be deleted (dry run)
git clean -f         # delete untracked files
git clean -fd        # also delete untracked folders
git clean -fdx       # also delete IGNORED files (careful: removes .env, venvs...)
git clean -i         # interactive mode
```

6.9 The reflog: your safety net

Git keeps a private diary of every place HEAD has been: every commit, checkout, reset, merge and rebase. This is the **reflog**. It is the reason almost nothing is ever lost.

```
git reflog
```

OUTPUT

```
2d08a7e (HEAD -> main) HEAD@{0}: reset: moving to HEAD~2
9c41f0b HEAD@{1}: commit: Add payment page
5e7a2c3 HEAD@{2}: commit: Add cart page
2d08a7e (HEAD -> main) HEAD@{3}: commit: Add product list
```

Oops: the reset threw away "Add payment page". The reflog still remembers it, so restore it:

```
git reset --hard 9c41f0b        # or: git reset --hard HEAD@{1}
```

Or rescue it onto a new branch without changing your current one:

```
git branch rescue 9c41f0b
```

The reflog is local only (it is not pushed), and old entries expire. By default, entries for commits that are no longer on any branch are kept for 30 days, and other entries for 90 days. So rescue lost work soon.

6.10 Quick decision table

I want to...	Command
Throw away edits in a file	`git restore <file>`
Unstage a file but keep edits	`git restore --staged <file>`
Change the last commit message (not pushed)	`git commit --amend -m "..."`
Add a forgotten file to the last commit (not pushed)	`git add <file>` then `git commit --amend --no-edit`
Undo the last commit, keep the work	`git reset --soft HEAD~1`
Undo the last commit and delete the work	`git reset --hard HEAD~1`
Undo a commit that is already pushed	`git revert <hash>`
Get one file back from an old commit	`git restore --source=<hash> <file>`
Delete untracked files	`git clean -n` then `git clean -fd`
Recover a "lost" commit	`git reflog` then `git branch rescue <hash>`
Temporarily put changes aside	`git stash` (next chapter)

CHAPTER 7

Stashing Work in Progress

7.1 Why stash?

You are halfway through a feature when an urgent bug report arrives. Your changes are not ready to commit, but you need a clean working tree to switch branches. **Stash** puts your uncommitted changes on a shelf and gives you a clean tree. Later you take them back.

```
git stash push -m "half-done search filter"
git switch main
# ... fix the bug, commit, push ...
git switch feature-search
git stash pop
```

`git stash` on its own (without `push -m`) also works; the message just makes it easier to find later.

7.2 Stash commands

Command	What it does
`git stash`	Stash tracked, modified files
`git stash push -m "msg"`	Stash with a description
`git stash -u`	Also stash untracked (new) files
`git stash -a`	Also stash ignored files
`git stash push -- path/file.py`	Stash only certain files
`git stash -p`	Choose hunks to stash interactively
`git stash list`	Show all stashes
`git stash show -p stash@{1}`	Show the diff inside a stash
`git stash pop`	Apply the newest stash and remove it from the list
`git stash apply stash@{2}`	Apply a stash but keep it in the list
`git stash drop stash@{0}`	Delete one stash
`git stash clear`	Delete all stashes
`git stash branch new-branch`	Create a branch from the stash's starting point and apply it there

```
OUTPUT
stash@{0}: On feature-search: half-done search filter
stash@{1}: WIP on main: 3a9f1e8 Initial commit
```

If `pop` causes a conflict, Git keeps the stash in the list. Resolve the conflict, then run `git stash drop` yourself.

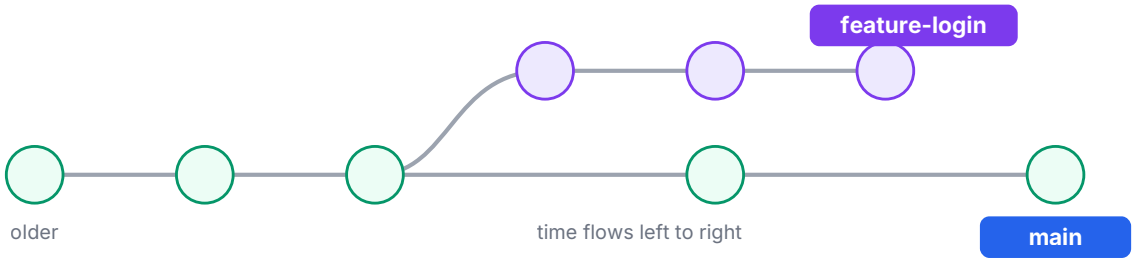
Stash or commit? Stashes are easy to forget, and a long list of nameless stashes is confusing. For anything longer than a quick interruption, many developers prefer a temporary commit on their feature branch: `git commit -m "WIP"`. Later they amend it or squash it away with an interactive rebase.

CHAPTER 8

Branches and Merging

8.1 Why branches matter

A **branch** is an independent line of work. You create one to build a feature, fix a bug or try a risky idea, without disturbing the stable version on `main`. If the idea works, you merge it back. If it fails, you delete the branch and nothing else is affected.



A feature branch splits from main. Both lines can receive new commits independently.

Remember from Chapter 3: a branch is only a pointer to a commit, so branches are cheap. Use them for everything, even one-line fixes.

8.2 Creating, listing and switching

```
git branch                      # list local branches (* marks the current one)
git branch -v                   # with the last commit of each
git branch -a                   # include remote-tracking branches
git branch -vv                  # show which remote branch each one tracks

git branch feature-login        # create a branch (but stay where you are)
git switch feature-login        # move to it
git switch -c feature-login     # create AND move in one step
git switch -c hotfix v1.2       # create a branch starting at a tag or commit
git switch -                    # go back to the previous branch
```

Before Git 2.23, `git checkout` did both switching branches and restoring files, which confused people. The modern split is: **`git switch`** for branches, **`git restore`** for files. You will still see the old forms often:

Old command	Modern equivalent
`git checkout feature`	`git switch feature`
`git checkout -b feature`	`git switch -c feature`
`git checkout -- file`	`git restore file`
`git checkout a1b2c3d -- file`	`git restore --source=a1b2c3d file`

When you switch branches, Git changes the files in your working tree to match that branch. If you have uncommitted changes that would be overwritten, Git refuses to switch and tells you. Commit or stash them first.

Renaming and deleting branches

```
git branch -m old-name new-name     # rename
git branch -m new-name              # rename the current branch
git branch -d feature-login         # delete (only if already merged)
git branch -D experiment            # force delete, even if not merged
```

Good branch names

Use short, lowercase names with hyphens, and a prefix that shows the purpose:

```
feature/user-registration
fix/login-crash-on-empty-password
docs/update-install-guide
refactor/split-payment-module
exp/try-resnet50-backbone
```

8.3 Detached HEAD

If you switch to a commit or tag instead of a branch, Git warns you that you are in **detached HEAD** state:

```
git switch --detach v1.0       # or the older: git checkout v1.0
```

HEAD now points directly at a commit, not at a branch. You can look around, build and test. You can even commit, but those commits belong to no branch, so they are easy to lose when you switch away. If you want to keep them, create a branch right there:

```
git switch -c fix-from-v1
```

8.4 Merging

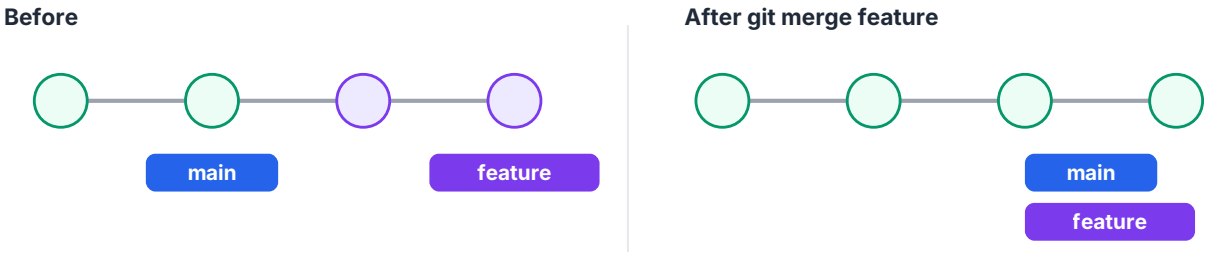
When work on a branch is finished, you combine it into another branch with `git merge`. You always merge **into** the branch you are currently on.

```
git switch main
git merge feature-login
```

Git chooses one of two strategies automatically.

Fast-forward merge

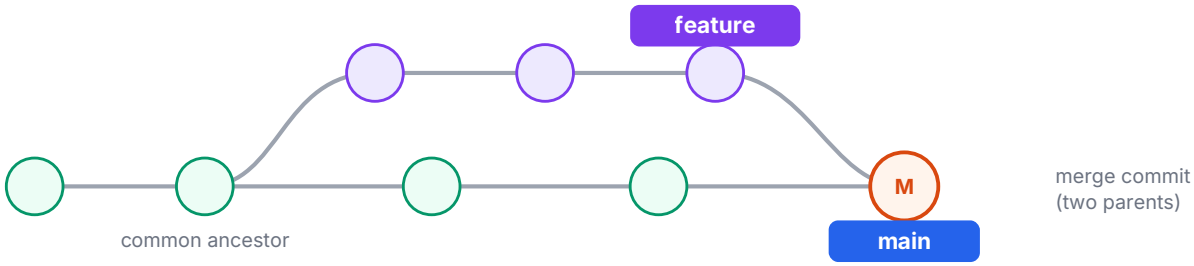
If `main` has not moved since the branch was created, Git simply moves the `main` pointer forward. No new commit is created.



Fast-forward: main just slides forward to the tip of feature. History stays a straight line.

Three-way merge

If both branches have new commits, Git looks at three snapshots: the tip of each branch and their common ancestor. It combines the changes and records a new **merge commit** with two parents.



Three-way merge: both lines moved, so Git creates a merge commit M that joins them.

```
OUTPUT
Merge made by the 'ort' strategy.
 login.py | 42 ++++++++++++++++++++++++++++++++++++++++
 1 file changed, 42 insertions(+)
```

Merge options

```
git merge --no-ff feature    # always create a merge commit, even if fast-forward is possible
git merge --ff-only feature  # only merge if fast-forward is possible; otherwise stop
git merge --squash feature   # combine all feature changes into staged edits, then commit once
git merge --abort            # give up a merge that has conflicts and go back
```

`--no-ff` keeps a visible record that a feature existed as a group of commits. `--squash` gives `main` one tidy commit per feature but loses the individual commits from the branch's history on `main`.

8.5 Merge conflicts

A **conflict** happens when two branches changed the *same lines* of the *same file* in different ways, or when one branch edited a file that the other deleted. Git cannot know which version is right, so it stops and asks you.

Conflicts are normal. They are not errors, and they do not mean you did anything wrong.

```
git merge feature-login
```

```
OUTPUT
Auto-merging config.py
CONFLICT (content): Merge conflict in config.py
Automatic merge failed; fix conflicts and then commit the result.
```

`git status` lists the conflicted files under "Unmerged paths". Open one and you will see **conflict markers**:

```
def connect():
<<<<<<< HEAD
    timeout = 30
    retries = 3
=======
    timeout = 60
>>>>>>> feature-login
    return open_connection(timeout)
```

- Between `<<<<<<< HEAD` and `=======` is the version on your current branch.
- Between `=======` and `>>>>>>> feature-login` is the version from the branch being merged.

Resolving step by step

1. **Edit the file** so it contains the correct final code. You may keep one side, the other, or combine both. Delete all three marker lines.

   ```
   def connect():
       timeout = 60
       retries = 3
       return open_connection(timeout)
   ```

2. **Test** that the code still works.
3. **Mark it resolved** by staging it: `git add config.py`
4. Repeat for every conflicted file, then **finish the merge**: `git commit` (Git prepares the message for you).

If things get confusing, `git merge --abort` returns everything to how it was before the merge.

Choosing one whole side

For a file where you simply want one side's version:

```
git checkout --ours   config.py    # keep the current branch's version
git checkout --theirs config.py    # keep the incoming branch's version
git add config.py
```

Better tools for conflicts

- **VS Code** highlights conflicts and offers buttons: *Accept Current*, *Accept Incoming*, *Accept Both*, plus a three-way merge editor.

- `git mergetool` opens a configured visual tool (Meld, KDiff3, P4Merge, Beyond Compare).
- Show the common ancestor inside the markers, which makes the right answer much easier to see:

```
git config --global merge.conflictStyle zdiff3
```

With this setting a third section, `||||||| base`, shows what the code looked like before either side changed it.

Avoiding painful conflicts

- Keep branches short-lived. Merge within days, not months.
- Pull or merge `main` into your branch regularly so differences stay small.
- Make small, focused commits.
- Agree on code formatting (use an auto-formatter such as Black or Prettier) so whitespace changes do not collide.
- Communicate: if two people must edit the same file heavily, talk first.

8.6 Cleaning up branches

```
git branch --merged main         # branches fully merged into main (safe to delete)
git branch --no-merged main      # branches with work not yet in main
git branch -d feature-login      # delete a merged branch
git push origin --delete feature-login   # delete it on the server too
```

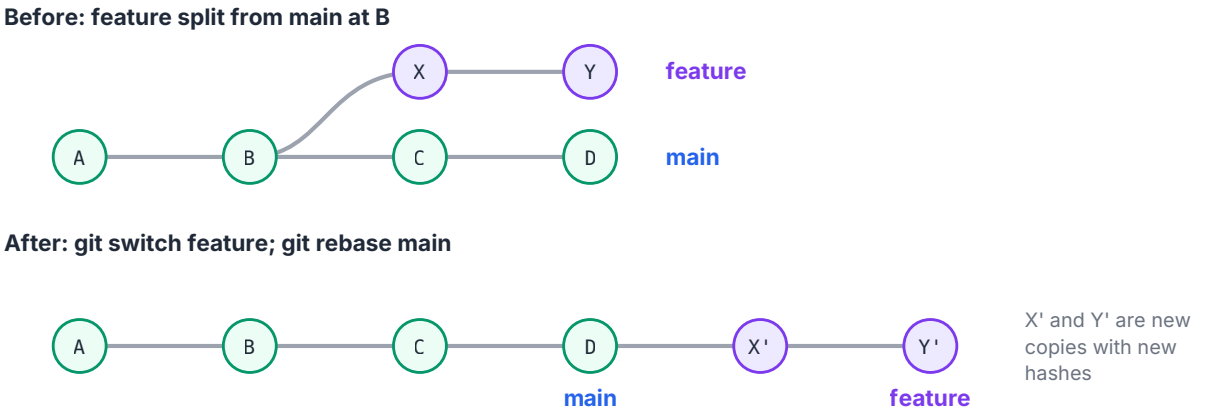
Try it: create a conflict on purpose. On `main`, make a file `colors.txt` containing `blue` and commit. Create a branch `paint` and change the line to `green`; commit. Switch back to `main`, change the same line to `red`; commit. Now run `git merge paint`. Read the conflict markers, resolve them to `purple`, stage and commit. Finally view the result with `git log --oneline --graph`.

CHAPTER 9

Rebasing and Rewriting History

9.1 What rebase does

Merging joins two lines of history with a merge commit. **Rebasing** takes a different approach: it moves your branch so it starts from the *new* tip of another branch, replaying your commits one by one on top.



Rebase replays X and Y on top of D. The result is a straight line, as if you had started your work later.

```
git switch feature
git rebase main
```

Now `main` can be fast-forwarded to include the feature, giving a clean, linear history:

```
git switch main
git merge --ff-only feature
```

Key point: rebase creates *new* commits (X' and Y'). They contain the same changes but have new hashes, because their parent is different. The old X and Y still exist for a while (the reflog can find them), but your branch no longer uses them.

9.2 Conflicts during a rebase

Because commits are replayed one at a time, you may resolve conflicts for each one:

```
# Git stops at a conflicting commit
# 1. fix the files, then:
git add <file>
git rebase --continue      # go on to the next commit

git rebase --skip          # drop this commit and continue
git rebase --abort         # cancel the whole rebase and go back
```

Do not run `git commit` during a rebase conflict; use `git rebase --continue`.

9.3 Merge or rebase?

	Merge	Rebase
History shape	True record, with branch lines and merge commits	Straight line, easy to read
Rewrites commits?	No	Yes (new hashes)
Safe on shared branches?	Yes	Only on branches nobody else uses
Conflict resolution	Once, for the whole merge	Possibly once per replayed commit
Best for	Integrating finished features into `main`	Updating your private branch with the latest `main`; tidying commits before sharing

Many teams combine both: **rebase your private feature branch** on `main` to keep it up to date and tidy, then **merge it** into `main` through a pull request.

9.4 The golden rule of rebasing

> **Never rebase commits that exist outside your repository and that other people may have based work on.** If you rebase commits that teammates already pulled, their history and yours will no longer match. They will see duplicate commits and confusing conflicts. Rebase only your own, unshared work, or a shared branch only after everyone agrees.

9.5 Interactive rebase: editing your history

Interactive rebase is one of Git's most powerful tools. Before you share a branch, you can clean it up: reword messages, combine small "fix typo" commits, reorder commits or delete one.

```
git rebase -i HEAD~4       # edit the last 4 commits
git rebase -i main         # edit every commit on this branch that is not on main
```

Git opens your editor with a to-do list, **oldest commit first**:

```
pick 1c7a5f2 Add login form
pick 8d2e4a1 Fix typo
pick 4b8e9d0 Add password strength meter
pick 6f0a3c7 WIP debugging

# Commands:
# p, pick   = use commit
# r, reword = use commit, but edit the commit message
# e, edit   = use commit, but stop for amending
# s, squash = use commit, but meld into previous commit
# f, fixup  = like "squash", but discard this commit's message
# d, drop   = remove commit
# x, exec   = run command (the rest of the line) using shell
```

Change the words at the start of the lines, reorder lines if you want, then save and close:

```
pick   1c7a5f2 Add login form
fixup  8d2e4a1 Fix typo
reword 4b8e9d0 Add password strength meter
drop   6f0a3c7 WIP debugging
```

Result: the typo fix is folded silently into "Add login form", Git lets you rewrite the meter commit's message, and the debugging commit disappears. Four messy commits become two clean ones.

The commands in plain words

Command	Effect
`pick`	Keep the commit as it is
`reword`	Keep the changes, edit the message
`edit`	Stop at this commit so you can change it (amend, split)
`squash`	Merge into the commit above and combine both messages
`fixup`	Merge into the commit above and keep only the above message
`drop`	Delete the commit (or just delete the line)
`exec`	Run a shell command, e.g. `exec pytest`, to test each step

Automatic fixups

When you notice a mistake in an earlier commit, make a "fixup" commit that points at it:

```
git add login.py
git commit --fixup=1c7a5f2
git rebase -i --autosquash main
```

Git automatically moves the fixup to the right place and marks it `fixup`. Set it as the default with `git config --global rebase.autoSquash true`.

Splitting one commit into two

Mark the commit as `edit` in the to-do list. When Git stops there:

```
git reset HEAD~1           # undo the commit but keep its changes as edits
git add -p                 # stage the first part
git commit -m "First part"
git add .
git commit -m "Second part"
git rebase --continue
```

9.6 Moving a branch to a new base: --onto

You created `feature-b` from `feature-a` by mistake, but it should start from `main`:

```
git rebase --onto main feature-a feature-b
```

Read it as: "take the commits of `feature-b` that are not in `feature-a`, and replay them onto `main`."

9.7 Pushing after a rebase

If you already pushed your branch and then rebased it, a normal push is rejected because the history changed. Use a *safe* force push:

```
git push --force-with-lease
```

`--force-with-lease` refuses to overwrite the remote branch if someone else pushed to it since you last fetched. Plain `--force` would silently destroy their work. Make `--force-with-lease` your habit and avoid plain `--force`.

Chapter summary

- Rebase replays your commits on a new base, producing a linear history with new commit hashes.
- Merge is always safe; rebase is safe only for commits that are not shared.
- `git rebase -i` lets you reword, squash, reorder, split and drop commits before sharing.
- `git commit --fixup` plus `--autosquash` automates cleanup.
- After rebasing a pushed branch, use `git push --force-with-lease`, never plain `--force`.

PART III

Remotes and GitHub

Sharing work with other people: remotes, authentication, repositories on GitHub, pull requests, issues, automation with Actions, releases, Pages and security.

CHAPTER 10

Working with Remotes

10.1 What is a remote?

A **remote** is another copy of your repository, usually on a server such as GitHub. Your local repository stores a short name for it and its URL. By convention:

- `origin` is the remote you cloned from, or your own copy on GitHub.
- `upstream` is the original project, when you work on a fork.

```
git remote -v                                    # list remotes with URLs
git remote add origin git@github.com:ayesha/shop.git
git remote rename origin github
git remote set-url origin git@github.com:ayesha/shop-v2.git
git remote remove old-server
git remote show origin                           # detailed information
```

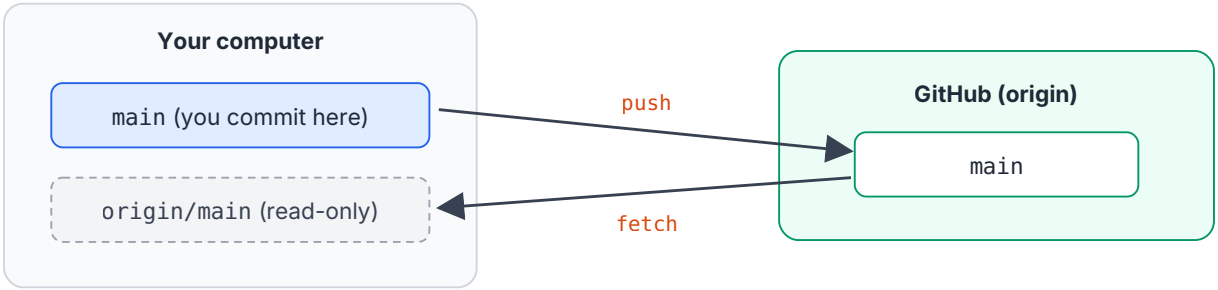
HTTPS or SSH URLs?

	HTTPS	SSH
Looks like	`https://github.com/user/repo.git`	`git@github.com:user/repo.git`
Login with	Personal access token or credential manager	SSH key
Works behind strict firewalls	Usually yes (port 443)	Sometimes blocked (port 22)
Setup effort	Low	One-time key setup

Both are fine. Chapter 11 shows how to set up each.

10.2 Remote-tracking branches

Your repository keeps a read-only memory of where each branch was on the remote the last time you talked to it. These are **remote-tracking branches**, named like `origin/main`.



fetch updates origin/main from the server; push sends your local main to the server.

`git status` compares your branch with its remote-tracking branch:

```
OUTPUT
On branch main
Your branch is ahead of 'origin/main' by 2 commits.
  (use "git push" to publish your local commits)
```

"Ahead" means you have commits the server does not. "Behind" means the server has commits you do not. "Diverged" means both.

> `origin/main` is only updated when you run `fetch`, `pull` or `push`. If a teammate pushed five minutes ago, `git status` will not know until you fetch.

10.3 fetch, pull and push

Command	What it does	Changes your files?
`git fetch`	Downloads new commits and updates `origin/*`	No. Completely safe.
`git pull`	`fetch`, then merges (or rebases) into your current branch	Yes
`git push`	Uploads your commits to the remote branch	No (changes the server)

git fetch: look before you leap

```
git fetch origin
git log --oneline main..origin/main      # what is new on the server?
git diff main origin/main                # how does it differ from mine?
git merge origin/main                    # bring it in when ready
```

Fetching first and then merging is the careful way. It lets you see what is coming.

git pull

```
git pull                    # fetch + merge (default)
git pull --rebase           # fetch + rebase your local commits on top
git pull --ff-only          # fetch + fast-forward only; stop if histories diverged
```

Choose a default once, or Git will ask you:

```
git config --global pull.rebase false   # merge (simple, safe)
git config --global pull.rebase true    # rebase (linear history)
git config --global pull.ff only        # refuse to merge or rebase automatically
```

Many experienced developers prefer `pull.rebase true` for feature branches: it avoids meaningless "Merge branch 'main' of github.com..." commits.

git push

```
git push -u origin main          # first push: also set origin/main as upstream
git push                         # later pushes: Git remembers the upstream
git push -u origin feature-x     # publish a new branch
git push origin --delete feature-x   # delete a branch on the server
git push origin v1.0.0           # push one tag
git push --tags                  # push all tags
git push --force-with-lease      # safe force push after a rebase
```

The `-u` (or `--set-upstream`) flag links your local branch to the remote one. After that, plain `git push` and `git pull` know where to go, and `git status` reports ahead and behind.

Set this once to make the first push of every new branch easier:

```
git config --global push.autoSetupRemote true
```

Now a plain `git push` on a new branch creates it on the server and sets the upstream automatically.

10.4 When a push is rejected

```
OUTPUT
 ! [rejected]        main -> main (fetch first)
error: failed to push some refs to 'github.com:ayesha/shop.git'
hint: Updates were rejected because the remote contains work that you do not
hint: have locally. This is usually caused by another repository pushing to
hint: the same ref.
```

Someone pushed before you. Git protects their work by refusing to overwrite it. The fix:

```
git pull            # or: git pull --rebase
# resolve any conflicts, then:
git push
```

> Do not "fix" a rejected push with `git push --force`. That deletes your teammate's commits from the server. Pull first, then push.

10.5 Keeping things tidy

When branches are deleted on GitHub, your `origin/...` copies remain until you prune them:

```
git fetch --prune
git config --global fetch.prune true     # prune automatically on every fetch
```

10.6 Several remotes

You can have as many remotes as you like. The most common case is a fork:

```
git remote add upstream https://github.com/original-owner/project.git
git fetch upstream
git switch main
git merge upstream/main         # bring the original project's changes into your main
git push origin main            # update your fork on GitHub
```

Chapter summary

- A remote is a named URL; `origin` is the usual default, `upstream` the original project of a fork.
- `origin/main` is your local, read-only memory of the server's `main`, updated by fetch, pull and push.
- `fetch` is always safe; `pull` is fetch plus merge or rebase; `push` uploads.
- A rejected push means the server has new work: pull, then push. Never force push over others.

CHAPTER 11

Getting Started with GitHub

11.1 Your account

1. Sign up at `github.com` with a username you are happy to show employers and professors. It appears in every URL of your work.
2. Add a profile photo, your name, a short bio and your affiliation.
3. **Turn on two-factor authentication (2FA)** under Settings, then Password and authentication. GitHub requires 2FA for most accounts that contribute code. Use an authenticator app or a passkey, and save your recovery codes somewhere safe.
4. Under Settings, then Emails, decide whether to keep your email private. If you do, use the `noreply` address in your Git configuration (Chapter 2).

Students and teachers can apply for free benefits through **GitHub Education** (`education.github.com`), including extra features and partner tools.

11.2 Authenticating Git with GitHub

GitHub does **not** accept your account password for Git operations over HTTPS. You must use one of these methods.

Option 1: SSH keys (recommended for your own computer)

Step 1. Create a key pair.

```
ssh-keygen -t ed25519 -C "you@example.com"
```

Press Enter to accept the default location. Choose a passphrase (recommended). This creates two files in `~/.ssh/` : `id_ed25519` (private, never share it) and `id_ed25519.pub` (public, safe to share).

Step 2. Load the key into the SSH agent so you do not type the passphrase every time.

```
eval "$(ssh-agent -s)"
ssh-add ~/.ssh/id_ed25519
```

On macOS, add `--apple-use-keychain` to `ssh-add` to store the passphrase in Keychain.

Step 3. Add the public key to GitHub. Copy the contents of the `.pub` file:

```
cat ~/.ssh/id_ed25519.pub
```

On GitHub, go to Settings, then SSH and GPG keys, then *New SSH key*. Paste the key, give it a name like "Lab PC", and save.

Step 4. Test the connection.

```
ssh -T git@github.com
```

```
OUTPUT
Hi ayesha! You've successfully authenticated, but GitHub does not provide shell access.
```

Now clone and push with SSH URLs: `git@github.com:user/repo.git`.

Option 2: HTTPS with a personal access token

A **personal access token (PAT)** is a long random password with limited permissions and an expiry date.

1. On GitHub: Settings, then Developer settings, then Personal access tokens.
2. Prefer a **fine-grained token**. Choose which repositories it can access, give it only the permissions it needs (for example, *Contents: Read and write*), and set an expiry date.
3. Copy the token once; GitHub will not show it again.
4. When Git asks for a password over HTTPS, paste the token.

So you do not paste it every time, use a credential helper. Git for Windows and GitHub Desktop include **Git Credential Manager**, which stores tokens securely and can open a browser login instead. On macOS, the `osxkeychain` helper does the same.

Option 3: GitHub CLI

The official command-line tool `gh` can set up authentication for you and does much more:

```
gh auth login          # follow the prompts; choose HTTPS or SSH
gh auth status
```

After `gh auth login`, Git over HTTPS works without further setup. You will meet more `gh` commands throughout this part.

> **Treat keys and tokens like passwords.** Never paste them in code, chat messages, screenshots or notebooks. If one leaks, delete it on GitHub immediately and create a new one.

11.3 Creating a repository on GitHub

Through the website

1. Click **+** (top right), then **New repository**.
2. Choose an owner (you or an organization) and a short name, such as `traffic-sign-classifier`.
3. Add a one-line description.
4. Choose **Public** (anyone can see) or **Private** (only people you invite).
5. Optionally add a README, a `.gitignore` template and a license.
6. Click **Create repository**.

Connecting an existing local project

If your project already exists on your computer, create an **empty** repository on GitHub (no README, no license), then:

```
cd my-project
git remote add origin git@github.com:ayesha/my-project.git
git branch -M main          # make sure the branch is called main
git push -u origin main
```

> If you created the GitHub repository *with* a README and also ran `git init` locally, the two histories are unrelated and your first push is rejected. Either create the GitHub repository empty, or clone it first and copy your files in. Chapter 20 shows the fix if it already happened.

With the GitHub CLI

```
# From inside an existing local repository:
gh repo create my-project --private --source=. --remote=origin --push

# Or create a new one and clone it:
gh repo create my-project --public --clone --add-readme
```

11.4 The README file

`README.md` is the front page of your repository. GitHub displays it below the file list. A good README answers:

1. **What** is this project, in one or two sentences?
2. **Why** does it exist, and who is it for?
3. **How** do I install and run it? (Exact commands.)
4. **How** do I use it? (A short example, a screenshot.)
5. **How** can I contribute, and under what **license**?
6. For research code: **how to cite** it, and which paper it accompanies.

README.MD

```
# Traffic Sign Classifier

A lightweight CNN that recognises 43 classes of traffic signs,
reaching 98.1% test accuracy on GTSRB.

## Installation
    git clone https://github.com/ayesha/traffic-sign-classifier.git
    cd traffic-sign-classifier
    pip install -r requirements.txt

## Usage
    python predict.py --image examples/stop.png

## Results
| Model     | Params | Accuracy |
|-----------|--------|----------|
| Baseline  | 1.2 M  | 96.4%    |
| Ours      | 0.9 M  | 98.1%    |

## License
MIT. See LICENSE.
```

11.5 Choosing a license

Without a license, nobody may legally reuse your code, even if the repository is public. GitHub offers a picker; `choosealicense.com` explains each one simply.

License	In short	Good for
MIT	Do almost anything; keep the copyright notice	Most small projects, research code
Apache 2.0	Like MIT, plus an explicit patent grant	Larger or company-backed projects
GPL 3.0	Derived works must also be open source under GPL	Projects that must stay open
BSD 3-Clause	Similar to MIT, with a no-endorsement clause	Academic and scientific software
CC BY 4.0	Reuse with credit (not meant for code)	Documentation, datasets, figures
No license	All rights reserved by default	Private work only

11.6 Repository settings worth knowing

- **About section** (gear icon on the main page): description, website and **topics** (tags like `computer-vision`, `pytorch`) that help people find your project.
- **Settings, then Collaborators:** invite people to push to a personal repository.
- **Settings, then General:** rename the repository, change the default branch, enable features (Issues, Discussions, Wiki, Projects), choose allowed merge methods, and in the *Danger Zone* change visibility, transfer or archive the repository.
- **Template repository:** tick this to let others start new repositories from yours with one click. Very useful for course starter code.

- **Organizations:** free shared accounts for teams, labs or clubs, with teams and fine-grained permissions.

11.7 Your profile README

Create a public repository with **exactly your username** as its name (for example `ayesha/ayesha`) and add a `README.md`. GitHub shows it at the top of your profile page. Use it to introduce your research interests, current projects, publications and contact links. It is often the first thing a recruiter or professor sees.

CHAPTER 12

Collaboration with Pull Requests

12.1 What is a pull request?

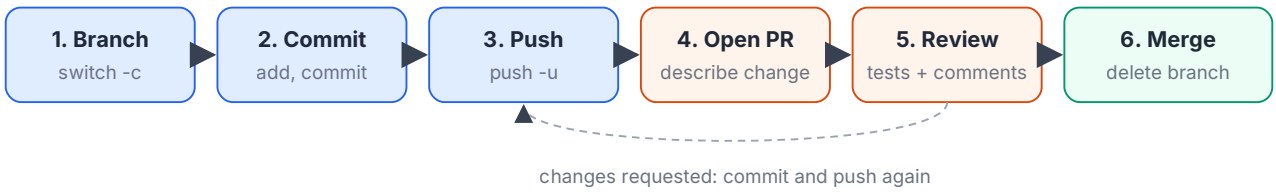
A **pull request (PR)** is a request to merge one branch into another, together with a page where people discuss and review the change. It is the heart of collaboration on GitHub. GitLab calls the same idea a *merge request*.

A PR page shows:

- **Conversation:** the description, comments and the timeline of events.
- **Commits:** the commits included.
- **Checks:** results of automated tests (GitHub Actions).
- **Files changed:** the diff, where reviewers can comment on specific lines.

12.2 The shared-repository workflow

Used when everyone on the team can push to the same repository (a company team, a lab, a student group).



The pull request cycle. Review often loops back to more commits before the merge.

```
# 1. Start from an up-to-date main
git switch main
git pull

# 2. Create a branch and do the work
git switch -c fix/cart-total-rounding
# ...edit files...
git add -p
git commit -m "Round cart total to two decimal places"

# 3. Publish the branch
git push -u origin fix/cart-total-rounding
```

After pushing, GitHub shows a yellow banner with a **Compare & pull request** button. Or use the CLI:

```
gh pr create --fill                    # title and body from your commits
gh pr create --draft --title "WIP: new checkout page"
gh pr view --web                       # open it in the browser
```

12.3 Writing a good pull request

- **Title:** short and specific, like a commit summary: "Round cart total to two decimal places".
- **Description:** what changed, why, and how to test it. Add screenshots for visual changes.
- **Link issues:** writing `Closes #42` (or `Fixes #42`, `Resolves #42`) in the description closes issue 42 automatically when the PR is merged into the default branch.
- **Keep it small.** Reviews of 50 to 300 changed lines get careful attention; reviews of 3,000 lines get a quick "looks good to me".
- **Draft PRs:** open a PR as a draft to share early work and get feedback before it is ready to merge.

EXAMPLE PR DESCRIPTION

```
## What
Rounds the cart total to two decimal places before display and payment.

## Why
Customers saw totals like 19.999999 because of floating point maths. Closes #42.

## How to test
1. Add three items priced 6.33 to the cart
2. Total should show 18.99, not 18.989999

## Notes
Uses Decimal instead of float for money values.
```

A repository can provide a default description with a file at `.github/pull_request_template.md`.

12.4 Reviewing a pull request

Open the **Files changed** tab. You can:

- Click the **+** next to a line to comment on it. Drag to select several lines.
- **Suggest a change** that the author can accept with one click:

````
```suggestion
total = round(subtotal + tax, 2)
```
````

- Submit your review as **Comment** (general feedback), **Approve** (ready to merge) or **Request changes** (must be fixed first).

Good review etiquette:

- Review the code, not the person. Prefer "This loop could miss the last item" to "You made an error".
- Explain *why*, and ask questions when unsure: "Could this be `None` here?"
- Separate must-fix issues from optional suggestions (many reviewers prefix optional ones with "nit:").

- Praise good work. Reviews are also for learning.
- As an author, reply to every comment, and do not take criticism personally.

Testing a PR on your own computer

```
gh pr checkout 57                    # easiest, with the CLI
# or with plain Git:
git fetch origin pull/57/head:pr-57
git switch pr-57
```

12.5 Updating a pull request

A PR follows its branch. To respond to review comments, simply commit and push to the same branch; the PR updates automatically.

```
# ...make the requested changes...
git commit -am "Use Decimal for tax calculation"
git push
```

If `main` has moved and your PR now has conflicts, update your branch:

```
git switch fix/cart-total-rounding
git fetch origin
git merge origin/main            # or: git rebase origin/main
# resolve conflicts, test, then:
git push                         # after a rebase: git push --force-with-lease
```

For small conflicts, GitHub also offers a **Resolve conflicts** button in the browser.

12.6 Merging: three options

The green merge button has three modes. Repository settings decide which are allowed.

Option	What happens on main	Pros	Cons
Create a merge commit	All PR commits plus a merge commit	Full, true history	History can look busy
Squash and merge	All PR commits combined into one new commit	Clean, one commit per PR	Individual commit details are lost on main
Rebase and merge	PR commits replayed onto main, no merge commit	Linear history, commits kept	Commits get new hashes

Squash and merge is a popular default for teams and student projects: every PR becomes one clear commit on `main`, and messy work-in-progress commits disappear.

After the merge

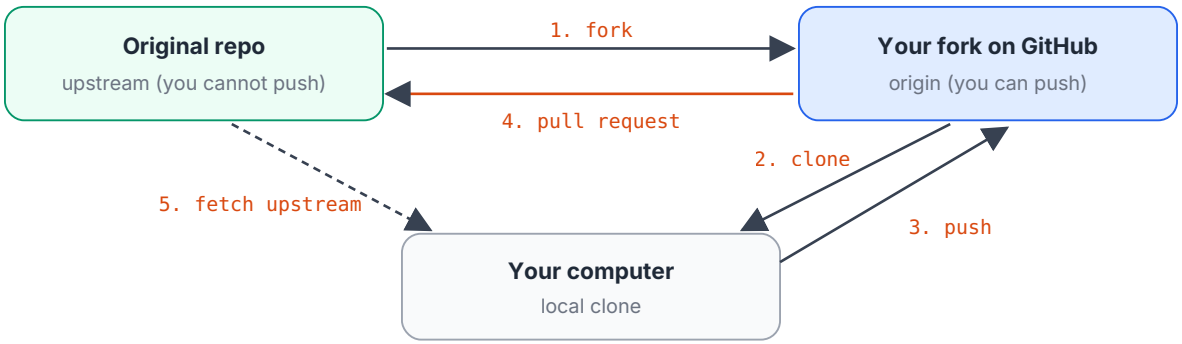
```
git switch main
git pull
git branch -d fix/cart-total-rounding     # delete the local branch
git fetch --prune                         # remove stale origin/* branches
```

Enable *Automatically delete head branches* in repository settings so GitHub removes merged branches on the server.

> After a **squash** merge, `git branch -d` may say the branch is "not fully merged", because its original commits are not on `main` (only the squashed copy is). If you are sure the PR was merged, use `git branch -D`.

12.7 The fork workflow (open source)

When you do not have permission to push to a project, you contribute through a **fork**: your own copy of the repository on GitHub.



The fork workflow: fork, clone, push to your fork, open a PR to the original, and keep in sync with upstream.

```
# 1. Click "Fork" on the project page (or: gh repo fork owner/project --clone)
# 2. Clone YOUR fork
git clone git@github.com:ayesha/project.git
cd project
# 3. Remember the original project
git remote add upstream https://github.com/owner/project.git
# 4. Work on a branch, never directly on main
git switch -c fix/typo-in-docs
git commit -am "Fix typo in installation guide"
git push -u origin fix/typo-in-docs
# 5. Open a PR from your fork's branch to the original's main
gh pr create --repo owner/project --fill
```

Keeping your fork up to date

```
git switch main
git fetch upstream
git merge --ff-only upstream/main
git push origin main
```

Or click **Sync fork** on your fork's GitHub page, then `git pull` locally. With the CLI, `gh repo sync ayesha/project` updates your fork on GitHub from the original.

Contributing to open source: etiquette

- Read `CONTRIBUTING.md` and the code of conduct first.
- For anything bigger than a typo, open or comment on an issue before writing code, so you do not build something the maintainers will not accept.
- Look for issues labelled `good first issue` or `help wanted`.
- Follow the project's style, run its tests, and keep PRs focused.
- Be patient. Maintainers are often volunteers.

CHAPTER 13

Issues, Projects and Communication

13.1 Issues

Issues track work: bugs, feature ideas, questions and tasks. Each issue gets a number (`#42`) that you can reference anywhere on GitHub.

A good bug report contains:

1. What you expected to happen.
2. What actually happened (exact error message, screenshot).
3. Steps to reproduce, as short as possible.
4. Environment: operating system, versions, browser.

Organizing issues

Feature	Use it for
Labels	Categories: `bug`, `enhancement`, `documentation`, `good first issue`, `priority: high`
Assignees	Who is responsible
Milestones	Group issues for a release or deadline, with a progress bar
Issue types	Organisation-wide types such as Bug, Feature and Task
Sub-issues	Break a large issue into smaller tracked pieces
Task lists	Checkboxes inside the issue body
Projects	Plan and track issues on boards and tables (next section)

Linking work together

- `#42` links to issue or PR 42 in the same repository.
- `owner/repo#42` links to an issue in another repository.
- `@username` notifies a person; `@org/team-name` notifies a team.
- A commit hash (`a1b2c3d`) becomes a link to that commit.
- `Closes #42` in a PR description or commit message closes the issue when merged into the default branch.

From the command line:

```
gh issue list --label bug
gh issue create --title "Login fails with empty password" --label bug
gh issue view 42 --web
gh issue close 42 --comment "Fixed in #57"
```

Issue templates

Templates make reporters give you the information you need. Put them in `.github/ISSUE_TEMPLATE/`. A YAML **issue form** creates proper input fields:

.GITHUB/ISSUE_TEMPLATE/BUG_REPORT.YML

```yaml
name: Bug report
description: Report something that does not work
labels: ["bug"]
body:
  - type: textarea
    id: what-happened
    attributes:
      label: What happened?
      description: Include the full error message.
    validations:
      required: true
  - type: textarea
    id: steps
    attributes:
      label: Steps to reproduce
      placeholder: "1. Go to...  2. Click...  3. See error"
  - type: input
    id: version
    attributes:
      label: Version or commit
```

13.2 GitHub Flavored Markdown

Everywhere you write on GitHub (README files, issues, PRs, comments, wikis) you use Markdown with some GitHub extras.

You type	You get
`# Title`, `## Section`	Headings
`**bold**`, `*italic*`, `~~strike~~`	**bold**, *italic*, strikethrough
`` `code` ``	Inline code
` ```python ` … ` ``` `	Code block with syntax colouring
`- item` or `1. item`	Lists
`- [ ] task` / `- [x] done`	Checkbox task lists
`[text](https://url)`	Link
`![alt text](image.png)`	Image
`> quote`	Block quote

You type	You get
`| a | b |` with `|---|---|`	Table
`$E = mc^2$` and `$$...$$`	Mathematics (LaTeX syntax)
```` ```mermaid ````	Diagrams drawn from text (flowcharts, sequence diagrams)
`<details><summary>More</summary>...</details>`	Collapsible section
`> [!NOTE]`, `> [!WARNING]`	Highlighted alert boxes

Images can be pasted or dragged straight into an issue or PR comment box; GitHub uploads them for you.

13.3 GitHub Projects

Projects are flexible planning boards built on top of issues and pull requests. A project can pull items from several repositories.

- **Views:** *Board* (Kanban columns like Todo, In Progress, Done), *Table* (spreadsheet-like), and *Roadmap* (timeline).
- **Custom fields:** priority, size, iteration (sprint), dates, text.
- **Automation:** for example, move an item to *Done* when its PR is merged, or add every new issue with a certain label automatically.
- **Insights:** charts such as burn-up of completed work.

Projects suit a student team planning a semester project, a lab planning experiments for a paper, or a course tracking weekly tasks.

13.4 Discussions, Wiki and notifications

- **Discussions** is a forum inside a repository, for questions, ideas and announcements that are not concrete tasks. It keeps the issue list focused on actionable work. Enable it in repository settings.
- **Wiki** is a set of simple documentation pages attached to the repository. For important documentation, many projects prefer a `docs/` folder in the main repository, because it is reviewed in PRs like code.
- **Watching** a repository (top right of its page) controls which notifications you get. Choose *Participating and @mentions* for most repositories to avoid overload.
- **Notifications inbox** (`github.com/notifications`) collects everything in one place. Mark items done as you would emails.

13.5 Useful GitHub shortcuts

- Press `?` on any GitHub page to see all keyboard shortcuts.
- Press `t` in a repository to search files by name.
- Press `.` (full stop) in a repository to open it in **github.dev**, a VS Code editor in your browser.

- Press `y` on a file view to change the URL to a permanent link to that exact version (useful when sharing a line of code).
- Add `#L10-L20` to a file URL to highlight lines 10 to 20.

CHAPTER 14

GitHub Actions: Automation and CI/CD

14.1 What GitHub Actions does

GitHub Actions runs automated jobs on GitHub's servers when something happens in your repository. Typical uses:

- **Continuous Integration (CI):** run tests and code-quality checks on every push and pull request.
- **Continuous Delivery (CD):** build and deploy a website or release a package when you tag a version.
- **Housekeeping:** label new issues, close stale ones, check links, rebuild a paper PDF.

Actions are free for public repositories. Private repositories get a monthly allowance of minutes depending on the account plan.

14.2 The vocabulary

Term	Meaning
Workflow	An automated process defined in a YAML file in `.github/workflows/`
Event	What starts the workflow: a push, a PR, a schedule, a button click
Job	A group of steps that run on one machine; jobs run in parallel by default
Step	A single shell command or a reusable action
Action	A reusable building block, such as `actions/checkout`
Runner	The virtual machine that runs a job (Ubuntu, Windows or macOS)

14.3 Your first workflow: run Python tests

Create this file, commit and push it:

.GITHUB/WORKFLOWS/CI.YML

```yaml
name: CI

on:
  push:
    branches: [main]
  pull_request:

permissions:
  contents: read

jobs:
  test:
    runs-on: ubuntu-latest
    strategy:
      matrix:
        python-version: ["3.11", "3.12", "3.13"]

    steps:
      - name: Get the code
        uses: actions/checkout@v7

      - name: Set up Python ${{ matrix.python-version }}
        uses: actions/setup-python@v6
        with:
          python-version: ${{ matrix.python-version }}
          cache: pip

      - name: Install dependencies
        run: |
          python -m pip install --upgrade pip
          pip install -r requirements.txt
          pip install pytest ruff

      - name: Check code style
        run: ruff check .

      - name: Run tests
        run: pytest -v
```

What this does:

- **`on`:** runs on every push to `main` and on every pull request.
- **`permissions`:** gives the workflow read-only access to the code (a good security habit).
- **`matrix`:** runs the same job three times, once per Python version, in parallel.
- **`uses`:** runs a published action. `actions/checkout` downloads your code; `actions/setup-python` installs Python and caches pip downloads.
- **`run`:** runs shell commands. A step that fails (non-zero exit code) fails the job.

Open the **Actions** tab to watch it run. On a pull request, the result appears in the *Checks* section with a green tick or a red cross. Click a failed step to read its log.

Action versions. The `@v7` and `@v6` suffixes select a major version of each action. Check each action's page on GitHub Marketplace for its current version, and let **Dependabot** (Chapter 16) keep them updated for you. For maximum security, large projects pin actions to a full commit hash instead of a version tag.

14.4 Common triggers

```yaml
on:
  push:
    branches: [main]
    paths: ["src/**", "tests/**"]     # only when these files change
  pull_request:
    branches: [main]
  schedule:
    - cron: "0 2 * * 1"               # every Monday at 02:00 UTC
  workflow_dispatch:                  # adds a "Run workflow" button
  release:
    types: [published]
```

14.5 Secrets and variables

Never write passwords or API keys in workflow files. Store them in **Settings, then Secrets and variables, then Actions**, and refer to them by name:

```yaml
    steps:
      - name: Deploy
        run: ./deploy.sh
        env:
          API_TOKEN: ${{ secrets.DEPLOY_TOKEN }}
```

GitHub hides secret values in logs. Secrets are not given to workflows triggered by pull requests from forks, so outsiders cannot steal them.

Every workflow also gets an automatic `GITHUB_TOKEN` with permissions you control through the `permissions:` key.

14.6 Saving build outputs: artifacts

A job's machine is deleted when the job ends. To keep a file, such as a built PDF, a trained model's metrics or a test report, upload it as an **artifact**:

```
      - name: Build paper
        run: latexmk -pdf paper/main.tex

      - name: Upload PDF
        uses: actions/upload-artifact@v7
        with:
          name: paper-pdf
          path: paper/main.pdf
```

The artifact appears on the workflow run page for download.

14.7 A status badge

Show the CI status at the top of your README:

```
![CI](https://github.com/ayesha/shop/actions/workflows/ci.yml/badge.svg)
```

14.8 Security and good practice

- Set `permissions:` to the minimum each workflow needs.
- Use well-known actions from trusted publishers, and review what third-party actions do.
- Be very careful with the `pull_request_target` and `workflow_run` events. They run with access to secrets, so never use them to run code from an untrusted pull request. Recent versions of `actions/checkout` add protections against this mistake.
- Do not print secrets, even partially.
- Keep workflows fast: cache dependencies and only run heavy jobs when relevant paths change.

14.9 Debugging failed workflows

1. Read the log of the failed step from the bottom up; the real error is usually near the end.
2. Reproduce locally with the same commands.
3. Use **Re-run jobs**, optionally *with debug logging*, from the run page.
4. Add a temporary step such as `run: ls -la && python --version` to inspect the machine.
5. The open source tool `act` can run many workflows on your own computer using Docker.

CHAPTER 15

Tags, Releases, Pages and More

15.1 Tags: naming important commits

A **tag** is a permanent name for a commit, usually a version number. Unlike a branch, it does not move.

```
git tag                                  # list tags
git tag -a v1.0.0 -m "First stable release"   # annotated tag (recommended)
git tag v1.0.0-rc1                       # lightweight tag (just a name)
git tag -a v0.9.0 a1b2c3d -m "Tag an older commit"
git show v1.0.0                          # details of a tag
git push origin v1.0.0                   # tags are NOT pushed automatically
git push --tags                          # push all tags
git tag -d v1.0.0                        # delete locally
git push origin --delete v1.0.0          # delete on GitHub
```

Annotated tags store the tagger, date and a message, and can be signed. Use them for releases. **Lightweight tags** are simple bookmarks for private use.

Semantic versioning

Most projects number versions as **MAJOR.MINOR.PATCH**, for example `2.4.1`:

Part	Increase it when...	Example
MAJOR	You make incompatible changes that can break users' code	`1.9.3` to `2.0.0`
MINOR	You add features in a backward-compatible way	`2.0.0` to `2.1.0`
PATCH	You make backward-compatible bug fixes	`2.1.0` to `2.1.1`

Versions below `1.0.0` signal that anything may still change. Suffixes like `-alpha`, `-beta.2` and `-rc.1` mark pre-releases.

15.2 GitHub Releases

A **release** is a GitHub page built on a tag. It adds release notes and downloadable files (installers, datasets, compiled binaries, model weights). Users can subscribe to new releases.

On the website: **Releases**, then **Draft a new release**, choose or create a tag, click **Generate release notes** to list merged PRs automatically, attach files, and publish.

With the CLI:

```
gh release create v1.0.0 --title "Version 1.0.0" --generate-notes
gh release create v1.1.0 dist/*.zip --notes-file CHANGELOG.md
gh release list
```

Releases are also how GitHub connects to **Zenodo** to give research software a citable DOI (Chapter 19).

15.3 GitHub Pages: free websites

GitHub Pages hosts static websites (HTML, CSS, JavaScript) directly from a repository. Ideal for personal academic homepages, project documentation, course websites and portfolios.

Two kinds of sites:

- **User site:** a repository named `username.github.io` is published at `https://username.github.io`.
- **Project site:** any other repository can be published at `https://username.github.io/repository-name`.

Publishing from a branch (simplest):

1. Put an `index.html` (or `index.md`) in the root of the repository or in a `docs/` folder.
2. Go to **Settings, then Pages**.
3. Under *Build and deployment*, choose *Deploy from a branch*, select `main` and `/ (root)` or `/docs`, and save.
4. After a minute the site is live. The Pages settings show its address.

Publishing with GitHub Actions lets you build the site first with any tool (Hugo, MkDocs, Quarto, Sphinx, Astro, a React app). Choose *GitHub Actions* as the source in Pages settings, and GitHub suggests ready-made workflows for popular tools.

You can also use your own domain (for example `www.yourname.com`) and enforce HTTPS from the same settings page.

> A Pages site is normally **public on the internet**, even when it is built from a private repository. Only some enterprise plans can restrict who sees it. Never publish private data on it.

15.4 Other GitHub features

- **Gists** (`gist.github.com`): share single files or small snippets. Every gist is a small Git repository you can clone.
- **github.dev:** press `.` in any repository for a quick browser editor. Good for small edits and reading code.
- **Codespaces:** a full cloud development environment (VS Code with a terminal) running on GitHub's machines, configured by a `.devcontainer` folder. Useful when students' computers differ or are slow.
- **GitHub Packages and Container Registry:** host Docker images and language packages next to your code.
- **Insights tab:** contributors, commit activity, traffic and the dependency graph of a repository.
- **GitHub Copilot:** AI assistance for writing code, explaining it and reviewing pull requests. Always review AI-generated code as carefully as code from a new teammate.
- **GitHub Mobile:** review PRs, reply to issues and read notifications from your phone.

CHAPTER 16

Security and Repository Protection

16.1 Protecting important branches

On a team, nobody should be able to push broken or unreviewed code straight into `main`. GitHub provides two ways to set rules: classic **branch protection rules** and the newer, more flexible **rulesets**. Find both under **Settings, then Rules / Branches**.

Useful rules for `main`:

Rule	Effect
Require a pull request before merging	No direct pushes; all changes go through PRs
Require approvals (e.g. 1 or 2)	Someone else must approve
Dismiss stale approvals	New commits require a fresh review
Require status checks to pass	CI must be green before merging
Require branches to be up to date	The PR must include the latest `main`
Require conversation resolution	All review comments must be resolved
Require linear history	Only squash or rebase merges are allowed
Require signed commits	Every commit must be verified
Block force pushes and deletions	Protects history from being rewritten or removed

16.2 CODEOWNERS

A `CODEOWNERS` file automatically requests reviews from the right people when certain files change:

.GITHUB/CODEOWNERS

```
# Default owners for everything
*                   @ayesha

# The data team owns the pipeline
/pipeline/          @lab-org/data-team

# Documentation
*.md                @rahim @nadia
```

Combined with a rule requiring code-owner review, this guarantees experts see changes to their area.

16.3 Dependabot

Most projects depend on many outside packages, and those packages sometimes have security holes. **Dependabot** watches your dependency files (`requirements.txt` , `package.json` , workflow files, and many more):

- **Dependabot alerts** warn you about known vulnerabilities in your dependencies.
- **Security updates** open pull requests that upgrade a vulnerable package to a fixed version.
- **Version updates** open regular PRs to keep dependencies current, configured with a file:

.GITHUB/DEPENDABOT.YML

```yaml
version: 2
updates:
  - package-ecosystem: "pip"
    directory: "/"
    schedule:
      interval: "weekly"
  - package-ecosystem: "github-actions"
    directory: "/"
    schedule:
      interval: "monthly"
```

16.4 Secret scanning and push protection

GitHub scans repositories for known types of secrets (cloud keys, tokens, private keys). With **push protection** on, GitHub blocks a push that contains a detected secret and explains how to remove it. This is available for public repositories and is worth enabling wherever you can (Settings, then *Code security*).

16.5 Code scanning

Code scanning with CodeQL analyses your code for security bugs such as SQL injection and reports them as alerts. It is free for public repositories, and you can enable a default setup with one click in *Code security* settings for supported languages.

16.6 Signing commits

Anyone can set `user.name` and `user.email` to anything, so a commit's author field can be faked. **Signing** proves who made a commit. GitHub shows a **Verified** badge on signed commits from known keys. The easiest method reuses your SSH key:

```
git config --global gpg.format ssh
git config --global user.signingkey ~/.ssh/id_ed25519.pub
git config --global commit.gpgsign true
git config --global tag.gpgsign true
```

Then on GitHub, add the same public key again under Settings, then SSH and GPG keys, choosing **Signing Key** as the key type. GPG keys work too, but take more setup.

16.7 Security checklist

- 2FA on for every account; recovery codes stored safely.
- Fine-grained tokens with expiry dates instead of long-lived classic tokens.
- `.env` and key files listed in `.gitignore` from the very first commit.
- Secret scanning and push protection enabled.
- Branch rules on `main`: PRs required, CI required, force pushes blocked.
- Dependabot alerts and updates enabled.
- Workflow `permissions:` set to the minimum.
- Collaborators given the lowest role that works (Read, Triage, Write, Maintain, Admin).
- A `SECURITY.md` file telling people how to report vulnerabilities privately.

PART IV

Professional Practice

How real teams organise their work, the advanced tools experts reach for, Git for research and teaching, and a recipe book for getting out of trouble.

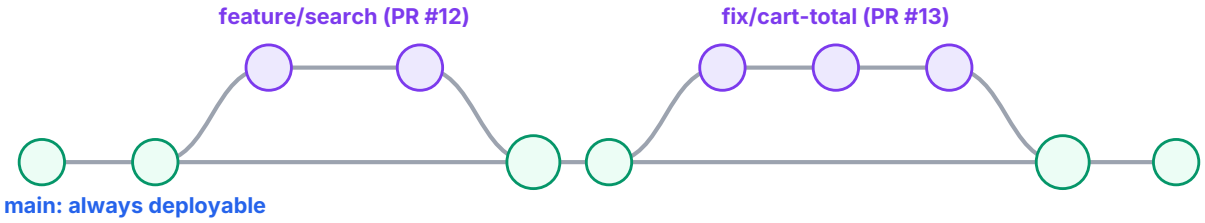
CHAPTER 17

Team Workflows and Conventions

Git does not force any particular way of working. Teams choose a **workflow**: an agreement about which branches exist, how work flows between them and how releases happen. Here are the most common ones.

17.1 GitHub Flow

The simplest and most popular workflow, ideal for web applications, student projects and most teams that deploy often.



GitHub Flow: one long-lived branch, short feature branches, pull requests for everything.

The rules:

1. `main` is always working and deployable.
2. For any change, create a descriptively named branch from `main`.
3. Commit to it and push it regularly.
4. Open a pull request for review and automated tests.
5. After approval and green checks, merge into `main`.
6. Deploy (often automatically) right after merging.

17.2 Git Flow

A more structured model with several long-lived branches. It suits software with planned, versioned releases (desktop apps, libraries, firmware) and the need to support old versions.

- `main`: only released, production versions, each tagged.
- `develop`: the integration branch for the next release.
- `feature/*`: branch from `develop`, merge back into `develop`.
- `release/*`: branch from `develop` to prepare a release (only fixes), then merge into `main` and `develop`.
- `hotfix/*`: branch from `main` for urgent production fixes, then merge into `main` and `develop`.

Git Flow is heavier than most teams need today. If you deploy continuously, GitHub Flow is usually better.

17.3 Trunk-based development

Everyone integrates into `main` (the "trunk") at least once a day, using very short-lived branches or even direct commits backed by strong automated tests. Unfinished features are hidden behind **feature flags** (configuration switches) rather than kept on long branches. Large tech companies favour this model because it avoids huge, painful merges.

17.4 Forking workflow

Contributors work in their own forks and send pull requests to the original repository (Chapter 12). Standard for open source, and useful for courses where each student must not push to the shared repository.

17.5 Which workflow should you choose?

Situation	Recommended
Solo project or coursework	GitHub Flow, or even commits on `main` plus branches for experiments
Student group project (3 to 6 people)	GitHub Flow with protected `main` and PR reviews
Web app or service, frequent deployment	GitHub Flow or trunk-based
Library or app with versioned releases and long support	Git Flow or GitHub Flow plus release branches
Open source with outside contributors	Forking workflow
Research lab code shared among members	GitHub Flow; tag versions used in papers

17.6 Conventional Commits

Conventional Commits is a simple, machine-readable format for commit messages. Tools can then generate changelogs and choose version numbers automatically.

```
<type>(<optional scope>): <short summary>

<optional body>

<optional footer, e.g. BREAKING CHANGE: or Closes #12>
```

Type	Use for	Example
`feat`	A new feature	`feat(auth): add password reset by email`
`fix`	A bug fix	`fix(cart): round total to two decimals`
`docs`	Documentation only	`docs: add installation steps for Windows`
`style`	Formatting, no code meaning change	`style: apply black formatting`
`refactor`	Restructure without changing behaviour	`refactor: split payment module`

Type	Use for	Example
`perf`	Performance improvement	`perf: cache product queries`
`test`	Adding or fixing tests	`test: cover empty cart case`
`build` / `ci`	Build system or CI changes	`ci: run tests on Python 3.13`
`chore`	Maintenance tasks	`chore: update dependencies`

A `!` after the type, as in `feat!: drop Python 3.8 support`, signals a breaking change (a MAJOR version bump under semantic versioning).

17.7 A day in the life of a team developer

```
# Morning: update
git switch main
git pull --rebase

# Pick issue #87 from the project board, start a branch
git switch -c feat/87-export-csv

# Work in small steps
git add -p
git commit -m "feat(report): add CSV export button"
git add -p
git commit -m "test(report): cover CSV export with unicode names"

# Main moved during the day? Bring it in
git fetch origin
git rebase origin/main

# Share and open a PR linked to the issue
git push -u origin feat/87-export-csv
gh pr create --fill --body "Closes #87"

# Respond to review, push fixes
git commit -am "fix(report): escape commas in CSV fields"
git push

# After it is merged
git switch main && git pull
git branch -D feat/87-export-csv
```

17.8 Team agreements worth writing down

Put these in `CONTRIBUTING.md` so everyone works the same way:

- Branch naming scheme and commit message format.
- Merge method (squash, merge commit or rebase) and who may merge.
- How many approvals a PR needs.
- Code formatter and linter (and a pre-commit hook to run them).
- How to run tests locally, and what "done" means: tests pass, docs updated, reviewed.

CHAPTER 18

Advanced Git Tools

18.1 Cherry-pick: copy a single commit

`git cherry-pick` applies the changes of one commit from somewhere else onto your current branch, as a new commit.

```
git switch release-1.2
git cherry-pick a1b2c3d             # copy one bug fix from main
git cherry-pick a1b2c3d e4f5a6b     # several commits
git cherry-pick -x a1b2c3d          # add "(cherry picked from ...)" to the message
git cherry-pick --abort             # if conflicts get messy
```

Typical use: a fix made on `main` must also go into an older release branch. Avoid cherry-picking as a substitute for proper merging; it duplicates commits.

18.2 Bisect: find the commit that broke things

Something worked last month and is broken today. There are 300 commits in between. `git bisect` performs a **binary search** through history: you mark versions as good or bad, and Git halves the range each time. 300 commits need at most about 9 tests.

```
git bisect start
git bisect bad                  # the current version is broken
git bisect good v2.0            # this older version was fine
# Git checks out a commit in the middle. Test it, then say:
git bisect good                 # or: git bisect bad
# ...repeat until Git prints "a1b2c3d is the first bad commit"
git bisect reset                # return to where you started
```

If you have a script that exits with 0 for good and non-zero for bad, Git can do everything automatically:

```
git bisect start HEAD v2.0
git bisect run pytest tests/test_checkout.py
```

18.3 Worktrees: several branches checked out at once

Instead of stashing and switching, check out another branch in a **separate folder** that shares the same repository:

```
git worktree add ../shop-hotfix hotfix/payment-crash
cd ../shop-hotfix          # work here while the main folder stays untouched
git worktree list
git worktree remove ../shop-hotfix
```

Very handy for reviewing a PR while you are in the middle of your own work, or for running a long training job on one branch while coding on another.

18.4 Submodules: a repository inside a repository

A **submodule** includes another Git repository at a specific commit, for example a shared library or a dataset tool.

```
git submodule add https://github.com/lab/common-utils.git libs/common-utils
git commit -m "Add common-utils as a submodule"

# Cloning a project that has submodules
git clone --recurse-submodules https://github.com/ayesha/project.git
# or, after a normal clone:
git submodule update --init --recursive

# Update a submodule to its latest version
cd libs/common-utils && git pull origin main && cd ../..
git add libs/common-utils
git commit -m "Update common-utils"
```

Submodules are powerful but easy to get wrong (people forget `--recursive`, or commit an outdated submodule pointer). Often a package manager dependency (`pip install git+https://...`) is simpler.

18.5 Git LFS: large files

Git stores every version of every file in full, so committing large binary files (videos, datasets, model weights, PSD files) makes repositories huge and slow. GitHub also rejects files over 100 MB. **Git Large File Storage (LFS)** keeps large files on a separate server and stores only small pointer files in Git.

```
git lfs install                    # once per computer
git lfs track "*.pt" "*.h5" "data/*.zip"
git add .gitattributes             # LFS records the patterns here
git add model.pt
git commit -m "Add trained model weights"
git push
git lfs ls-files                   # see which files are in LFS
```

GitHub gives a limited free LFS storage and bandwidth quota; check your account's billing settings. For very large research data, dedicated services are usually better (Chapter 19).

18.6 Hooks: run scripts automatically

Hooks are scripts Git runs at certain moments, stored in `.git/hooks/`. Examples: `pre-commit` (before a commit is created), `commit-msg` (to check the message), `pre-push` (before pushing). A hook that exits with an error stops the operation.

Because `.git/hooks/` is not shared through the repository, teams use the **pre-commit** framework to share hooks:

.PRE-COMMIT-CONFIG.YAML

```yaml
repos:
  - repo: https://github.com/pre-commit/pre-commit-hooks
    rev: v5.0.0
    hooks:
      - id: trailing-whitespace
      - id: end-of-file-fixer
      - id: check-added-large-files
      - id: detect-private-key
  - repo: https://github.com/astral-sh/ruff-pre-commit
    rev: v0.8.0
    hooks:
      - id: ruff
      - id: ruff-format
```

```
pip install pre-commit
pre-commit install            # sets up the Git hook
pre-commit autoupdate         # update the rev: versions to the latest
pre-commit run --all-files    # check the whole project once
```

Now every commit is automatically checked for formatting problems, large files and accidentally committed private keys. Run `pre-commit autoupdate` after copying this example, because the version numbers shown will be out of date.

18.7 .gitattributes: per-file behaviour

.GITATTRIBUTES

```
# Normalise line endings for all text files
* text=auto

# Always use LF for shell scripts, CRLF for Windows batch files
*.sh   text eol=lf
*.bat  text eol=crlf

# Treat these as binary (no diff, no line-ending changes)
*.png  binary
*.pdf  binary

# Large files stored in Git LFS
*.pt filter=lfs diff=lfs merge=lfs -text

# Hide generated files from GitHub's diff and language statistics
package-lock.json  linguist-generated=true
```

18.8 Big repositories: shallow clones and sparse checkout

```
git clone --depth 1 https://github.com/big/project.git        # only the latest commit
git clone --filter=blob:none https://github.com/big/project.git
# history now, file contents on demand

git sparse-checkout set src/frontend docs                     # only check out some folders
git sparse-checkout disable
```

Shallow clones are perfect for CI jobs and for quickly trying a project. Partial clones (`--filter`) keep the full history while downloading file contents only when needed.

18.9 rerere: remember conflict resolutions

If you resolve the same conflict repeatedly (for example while rebasing a long branch several times), let Git remember your solution:

```
git config --global rerere.enabled true
```

"rerere" means *reuse recorded resolution*. Next time the identical conflict appears, Git resolves it the same way automatically.

18.10 Ignoring formatting commits in blame

When a team reformats the whole codebase, `git blame` points every line at that formatting commit. List such commits in a file so blame skips them:

```
echo "a1b2c3d4e5f6a7b8c9d0e1f2a3b4c5d6e7f8a9b0" >> .git-blame-ignore-revs
git config blame.ignoreRevsFile .git-blame-ignore-revs
```

GitHub's blame view reads `.git-blame-ignore-revs` automatically.

18.11 Rewriting a whole history: git filter-repo

To remove a file from **every** commit (a leaked secret, a huge dataset), use `git filter-repo`, the tool recommended by the Git project:

```
pip install git-filter-repo
git filter-repo --path secrets.env --invert-paths      # delete a file everywhere
git filter-repo --strip-blobs-bigger-than 50M          # delete all huge files
```

This rewrites every commit hash. Everyone must re-clone afterwards, and you must force push. See Chapter 20 before using it.

18.12 Other handy commands

Command	Use
`git archive --format=zip -o code.zip HEAD`	Export a clean zip of the code, without the `.git` folder
`git describe --tags`	Name the current commit relative to the latest tag, e.g. `v1.2.0-5-ga1b2c3d`
`git range-diff`	Compare two versions of a branch, e.g. before and after a rebase
`git notes add -m "..."`	Attach a note to a commit without changing it
`git maintenance start`	Schedule background optimisation of a large repository
`git switch --orphan gh-pages`	Start a branch with no history (for a separate site or data)
`git log --show-signature`	Check commit signatures

CHAPTER 19

Git for Research and Teaching

Git is not only for software companies. In research it protects months of work, documents exactly how results were produced, and makes collaboration with co-authors far easier. In teaching it gives students an industry skill and gives teachers a clear view of each student's work.

19.1 A clean structure for a research repository

```
traffic-sign-study/
├── README.md              what, why, how to reproduce, how to cite
├── LICENSE
├── CITATION.cff           machine-readable citation
├── environment.yml        or requirements.txt, with pinned versions
├── .gitignore             data/raw, checkpoints, outputs, .env
├── data/
│   ├── README.md          where to download the data, and its licence
│   └── raw/               (ignored) original data, never edited
├── notebooks/             exploration, numbered: 01_eda.ipynb, 02_baseline.ipynb
├── src/                   reusable code: datasets.py, models.py, train.py
├── configs/               experiment settings: baseline.yaml, ablation_a.yaml
├── scripts/               run_all_experiments.sh, make_figures.py
├── results/               small final tables and figures (tracked)
└── paper/                 LaTeX source of the manuscript
```

Principles:

- Code, configuration and small results go in Git. Raw data, large checkpoints and temporary outputs do not.
- Raw data is read-only. Scripts create everything else from it.
- Each experiment is defined by a config file plus a commit. "Which code produced Table 3?" should have an exact answer.

19.2 Making results reproducible

1. **Record the commit with every result.** Save the output of `git rev-parse HEAD` into each experiment's log or results file:

   ```python
   import subprocess
   commit = subprocess.check_output(["git", "rev-parse", "HEAD"]).decode().strip()
   dirty = subprocess.call(["git", "diff", "--quiet"]) != 0
   print(f"code version: {commit}{' (uncommitted changes!)' if dirty else ''}")
   ```

2. **Do not run final experiments with uncommitted changes.** Commit first, then run, so the hash really describes the code.
3. **Pin your environment** (`pip freeze > requirements.txt` or `conda env export > environment.yml`) and commit it.

4. **Tag milestones:** `git tag -a paper-submitted-2026-10 -m "Code for the submitted version"`, then later `camera-ready` and `revision-1`. Reviewers' questions months later become easy to answer.
5. **Fix random seeds** in code and record them in configs.

19.3 Jupyter notebooks and Git

Notebooks (`.ipynb`) are JSON files containing code, outputs, images and execution counts. Their diffs are huge and unreadable, and outputs can leak data. Solutions:

- **Clear outputs before committing.** The `nbstripout` tool does it automatically:

```
pip install nbstripout
nbstripout --install          # installs a Git filter for this repository
```

- **Readable diffs and merges** with `nbdime` (`nbdiff`, `nbmerge`, and a web diff view). GitHub also shows rich diffs for notebooks in pull requests.
- **Pair notebooks with scripts** using `jupytext`, which keeps a `.py` version in sync. Review and diff the `.py` file.
- Move reusable code out of notebooks into `src/` and import it. Keep notebooks for exploration and figures.

19.4 Data: what to do with it

Data size and type	Approach
Small (a few MB), public, rarely changing	Commit it directly
Medium binary files (models, images), changes sometimes	Git LFS (Chapter 18)
Large or frequently changing datasets	DVC (Data Version Control) or similar, with data in cloud or lab storage
Public datasets owned by others	Do not copy them; give download scripts and links in `data/README.md`
Sensitive data (patients, personal information)	**Never** in Git, not even in a private repository. Follow your ethics approval and institution rules

> Private repositories are not a safe place for confidential or personal data. Collaborators, future team members, forks and backups all receive the full history, forever. Removing data later requires rewriting history on every copy.

19.5 Writing papers with Git

- **One sentence per line** in LaTeX or Markdown. Diffs then show exactly which sentences changed, and co-authors' edits merge without conflicts.
- Ignore build files (`*.aux`, `*.log`, `*.bbl`, `*.synctex.gz`, `*.out`), and decide whether to commit the final PDF (usually only at milestones or as a release asset).

- Use `latexdiff` to produce a "changes highlighted" PDF between two tagged versions for reviewers:

```
git show submitted:paper/main.tex > old.tex
latexdiff old.tex paper/main.tex > diff.tex
```

- **Overleaf** offers Git access and GitHub synchronisation on some plans, so co-authors can use the web editor while the history lives in Git.
- A GitHub Actions workflow can compile the PDF on every push and upload it as an artifact (Chapter 14), so everyone always has the latest version.

19.6 Making research software citable

CITATION.cff

Add a `CITATION.cff` file to the repository root. GitHub then shows a **Cite this repository** button with APA and BibTeX formats.

CITATION.CFF

```yaml
cff-version: 1.2.0
message: "If you use this software, please cite it as below."
title: "Traffic Sign Study: Code for Lightweight CNN Recognition"
authors:
  - family-names: "Rahman"
    given-names: "Ayesha"
    orcid: "https://orcid.org/0000-0000-0000-0000"
version: 1.0.0
date-released: 2026-10-20
url: "https://github.com/ayesha/traffic-sign-study"
license: MIT
```

A DOI through Zenodo

Zenodo, a free research repository run by CERN, integrates with GitHub. Log in to Zenodo with GitHub, switch on your repository, and every GitHub **Release** you publish is archived with its own **DOI**. Cite the DOI in your paper so readers get the exact code version, even if the GitHub repository later changes or disappears.

Public or private?

A common pattern: develop in a private repository; when the paper is accepted (or at preprint time), clean up the history if needed, add the README, license and citation file, and make it public. Many venues and reviewers now expect a code link.

19.7 Git in the classroom

Course repositories

- Publish lecture notes, code examples and lab sheets in a public course repository, organised by week (`week-01/`, `week-02/`, ...), with a README index.

- Use **Releases** or tags to mark the state of materials at the end of each semester.
- Use **Issues** or **Discussions** as a Q&A forum, so answers help the whole class.
- Turn on GitHub Pages to give the course a simple website from the same repository.

Assignments and group projects

- A **template repository** gives every student or team identical starter code with one click.
- **GitHub Classroom**, where available to your institution, automates creating a private repository per student or team from a template, collecting submissions at a deadline and running autograding tests with GitHub Actions.
- For group projects, require **pull requests and reviews** between teammates. It teaches professional practice and makes contributions visible.
- Assess contribution fairly using several signals: `git shortlog -sn`, the *Insights, then Contributors* graph, PR reviews and issue discussions. Commit counts alone mislead, because one large commit and ten tiny commits can represent the same effort.

A suggested learning path for students

1. Week 1: install, configure, `init`, `add`, `commit`, `status`, `log`, `.gitignore`.
2. Week 2: GitHub account, SSH key, `clone`, `push`, `pull`, README in Markdown.
3. Week 3: branches, merging, resolving a planned conflict in pairs.
4. Week 4: pull requests and code review in teams; issues and a project board.
5. Week 5: undoing things, `revert`, `reset`, `reflog`; a "rescue the repository" challenge.
6. Later: rebase, GitHub Actions for automated tests, releases and Pages.

The exercises in Appendix C follow this order.

CHAPTER 20

Troubleshooting and Recovery

A recipe book for common problems. Before trying any fix, take a breath and run:

```
git status
git log --oneline --graph --all -15
```

These two commands explain most situations. And remember the reflog: committed work is very hard to lose.

> **Make a safety copy first.** Before attempting a risky repair, create a backup branch pointing at your current state: `git branch backup-before-fix`. If the repair goes wrong, `git reset --hard backup-before-fix` brings you back.

20.1 "I committed to the wrong branch" (not pushed yet)

You made a commit on `main` that should be on a new branch:

```
git branch feature/new-thing       # new branch keeps the commit
git reset --hard HEAD~1            # move main back one commit
git switch feature/new-thing       # continue there
```

If the right branch already exists:

```
git switch correct-branch
git cherry-pick main               # copy the commit
git switch main
git reset --hard HEAD~1            # remove it from main
```

20.2 "I need to undo a merge or pull"

```
# Not pushed: return to the state before the merge
git reset --hard ORIG_HEAD

# Already pushed: create a commit that undoes the merge
git revert -m 1 <merge-commit-hash>
```

`ORIG_HEAD` is set by merge, rebase, reset and pull to where HEAD was before the operation.

20.3 "Push rejected: fetch first / non-fast-forward"

The remote has commits you do not have. Never force push to fix this on a shared branch.

```
git pull --rebase       # or: git pull
git push
```

20.4 "fatal: refusing to merge unrelated histories"

Typical cause: you created a repository on GitHub *with* a README, and separately ran `git init` locally. The two histories share no commit.

```
git pull origin main --allow-unrelated-histories
# resolve any conflict (often in README.md), commit, then:
git push -u origin main
```

20.5 "Permission denied (publickey)"

SSH cannot authenticate you.

```
ssh -T git@github.com          # test the connection
ssh-add -l                     # is a key loaded in the agent?
ssh-add ~/.ssh/id_ed25519      # load it
```

Check that the **public** key is listed in your GitHub settings, and that the remote URL is correct with `git remote -v`. If you are on a network that blocks port 22, GitHub supports SSH over port 443 through `ssh.github.com`, or you can switch to an HTTPS URL.

20.6 "Authentication failed" or "Support for password authentication was removed"

You typed your GitHub password for an HTTPS remote. Use a personal access token as the password, or run `gh auth login`, or switch to SSH:

```
git remote set-url origin git@github.com:user/repo.git
```

If a stored, expired token is being reused, remove it from your credential manager (Windows Credential Manager, macOS Keychain) and try again.

20.7 "Large files detected / file exceeds GitHub's 100 MB limit"

GitHub warns about files over 50 MB and blocks files over 100 MB. Deleting the file in a new commit is **not** enough, because it is still in an earlier commit.

```
# If the big file is only in the last commit (not pushed):
git rm --cached data/huge.csv
echo "data/huge.csv" >> .gitignore
git add .gitignore
git commit --amend --no-edit

# If it is deeper in unpushed history:
git filter-repo --path data/huge.csv --invert-paths
```

Then use Git LFS or external storage for such files (Chapters 18 and 19).

20.8 "I committed a password, key or token"

Step 1, always: revoke and replace the secret immediately (delete the token on GitHub, rotate the cloud key, change the password). Assume it is already compromised if it was pushed.

Step 2: remove it from history.

```
# Not pushed, in the last commit:
git rm --cached .env
git commit --amend --no-edit

# Pushed, or deeper in history: rewrite all commits
git filter-repo --path .env --invert-paths
# or replace the text everywhere:
#   echo 'MY_SECRET_VALUE==>REMOVED' > replacements.txt
#   git filter-repo --replace-text replacements.txt
git push --force-with-lease --all
```

Every collaborator must then re-clone. Rewriting history alone is not enough: forks, clones and caches may still hold the old commits, which is exactly why step 1 matters most.

20.9 "I am in detached HEAD and made commits"

```
git switch -c save-my-work          # creates a branch at your commits
```

If you already switched away, find the commits in the reflog and create a branch at them:

```
git reflog
git branch save-my-work <hash>
```

20.10 "I deleted a branch by mistake"

Git told you the hash when you deleted it:

```
OUTPUT
Deleted branch experiment (was 4f2a9c1).
```

```
git branch experiment 4f2a9c1
```

If the message is gone, search `git reflog` for the last commit you made on that branch.

20.11 "I ran git reset --hard and lost commits"

```
git reflog                          # find the commit before the reset
git reset --hard HEAD@{1}           # or the specific hash
```

Uncommitted changes discarded by `reset --hard` cannot be recovered by Git. Your editor's local history (VS Code has *Timeline*) is the last hope.

20.12 "I dropped a stash"

Dropped stashes are dangling commits until garbage collection removes them:

```
git fsck --no-reflogs | grep "dangling commit"
git show <hash>                     # inspect candidates
git stash apply <hash>              # restore the right one
```

20.13 "Wrong name or email on my commits"

```
# Fix configuration for the future
git config --global user.email "correct@example.com"

# Fix the last commit
git commit --amend --reset-author --no-edit

# Fix the last 5 commits (not pushed, or force push afterwards)
git rebase -r HEAD~5 --exec "git commit --amend --no-edit --reset-author"
```

20.14 "Remove one file from the last commit"

```
git restore --source=HEAD~1 --staged path/to/file    # unstage it from the commit
git commit --amend --no-edit
# the file's changes remain in your working tree
```

20.15 "Every file shows as modified, but I changed nothing"

Usually line endings or file permissions:

```
git diff --stat                     # see what Git thinks changed
git config core.autocrlf true       # Windows (input on macOS/Linux)
git config core.fileMode false      # ignore permission changes (e.g. on shared drives)
```

Adding a `.gitattributes` with `* text=auto` and running `git add --renormalize .` fixes line endings permanently.

20.16 "My fork is many commits behind"

```
git fetch upstream
git switch main
git merge --ff-only upstream/main
git push origin main
```

20.17 "Git asks me to write a message in a strange editor"

That is Vim. Press `Esc`, type `:wq` and press Enter to save and quit, or `:q!` to quit without saving. Then set a friendlier editor:

```
git config --global core.editor "code --wait"     # or "nano"
```

20.18 "A merge or rebase went wrong and I am lost"

```
git merge --abort
git rebase --abort
git cherry-pick --abort
git revert --abort
```

Each returns you to the state before the operation started.

20.19 When nothing else helps

- Read the message carefully. Git's hints usually tell you the next command.
- Turn on tracing to see what Git is doing: `GIT_TRACE=1 git push`.
- Copy your project folder somewhere safe before experimenting further.
- Search the exact error message. You are almost never the first person to see it.
- As a last resort, clone a fresh copy from GitHub and copy your changed files into it.

APPENDIX A

Command Cheat Sheet

Setup

Command	Purpose
`git config --global user.name "Name"`	Set your name
`git config --global user.email "you@x.com"`	Set your email
`git config --global init.defaultBranch main`	Default branch name for new repositories
`git config --global core.editor "code --wait"`	Choose your editor
`git config --list --show-origin`	Show all settings and where they come from

Start a project

Command	Purpose
`git init`	Make the current folder a repository
`git clone <url> [folder]`	Copy a repository with full history
`gh repo create <name> --source=. --push`	Create a GitHub repository from a local one

Everyday work

Command	Purpose
`git status` / `git status -s`	What has changed?
`git add <file>` / `git add .`	Stage changes
`git add -p`	Stage chosen parts of files
`git commit -m "message"`	Save a snapshot
`git commit --amend`	Change the last commit (not pushed)
`git rm <file>` / `git mv <a> <b>`	Delete / rename and stage
`git rm --cached <file>`	Stop tracking, keep on disk

Inspect

Command	Purpose
`git diff` / `git diff --staged`	Unstaged / staged changes
`git log --oneline --graph --all`	History graph
`git log -p <file>`	History of a file with diffs
`git log -S "text"`	Commits that added or removed text
`git show <commit>`	One commit in detail

Command	Purpose
`git show <commit>:<file>`	A file as it was in a commit
`git blame <file>`	Who last changed each line
`git grep "text"`	Search tracked files

Undo

Command	Purpose
`git restore <file>`	Discard unstaged edits (permanent)
`git restore --staged <file>`	Unstage, keep edits
`git restore --source=<commit> <file>`	Bring back an old version of a file
`git reset --soft HEAD~1`	Undo last commit, keep changes staged
`git reset HEAD~1`	Undo last commit, keep changes unstaged
`git reset --hard HEAD~1`	Undo last commit and delete its changes
`git revert <commit>`	Undo a pushed commit with a new commit
`git clean -n` / `git clean -fd`	Preview / delete untracked files
`git reflog`	Find "lost" commits

Stash

Command	Purpose
`git stash push -m "msg"`	Shelve uncommitted changes
`git stash -u`	Include untracked files
`git stash list`	List stashes
`git stash pop` / `apply`	Restore (and remove / keep)
`git stash drop` / `clear`	Delete one / all

Branches

Command	Purpose
`git branch` / `git branch -a`	List local / all branches
`git switch <branch>`	Move to a branch
`git switch -c <branch>`	Create and move
`git branch -m <new>`	Rename current branch
`git branch -d <branch>` / `-D`	Delete merged / force delete
`git merge <branch>`	Merge into the current branch
`git merge --abort`	Cancel a conflicted merge

Command	Purpose
`git rebase <base>`	Replay current branch onto base
`git rebase -i HEAD~n`	Edit the last n commits
`git rebase --continue` / `--abort`	Continue / cancel a rebase
`git cherry-pick <commit>`	Copy a commit onto this branch

Remotes

Command	Purpose
`git remote -v`	List remotes
`git remote add <name> <url>`	Add a remote
`git fetch` / `git fetch --prune`	Download, do not merge / also prune
`git pull` / `git pull --rebase`	Fetch and merge / rebase
`git push -u origin <branch>`	First push of a branch
`git push`	Upload commits
`git push --force-with-lease`	Safe force push after a rebase
`git push origin --delete <branch>`	Delete a remote branch

Tags and releases

Command	Purpose
`git tag -a v1.0.0 -m "msg"`	Create an annotated tag
`git push origin v1.0.0` / `--tags`	Push a tag / all tags
`gh release create v1.0.0 --generate-notes`	Publish a GitHub release

GitHub CLI

Command	Purpose
`gh auth login`	Authenticate
`gh repo clone owner/repo`	Clone
`gh repo fork owner/repo --clone`	Fork and clone
`gh pr create --fill`	Open a pull request
`gh pr checkout <number>`	Test a PR locally
`gh pr merge --squash --delete-branch`	Merge a PR
`gh issue create` / `gh issue list`	Work with issues
`gh run list` / `gh run watch`	Follow GitHub Actions runs

APPENDIX B

Glossary

Term	Meaning
Action	A reusable step for GitHub Actions workflows, such as `actions/checkout`.
Ahead / behind	How many commits your branch has that its upstream lacks, and the reverse.
Blob	Git's stored content of one file version.
Branch	A movable pointer to a commit; an independent line of development.
Checkout	Older command for switching branches and restoring files; now split into `switch` and `restore`.
Cherry-pick	Copy the changes of one commit onto the current branch.
CI/CD	Continuous Integration / Continuous Delivery: automatic testing and deployment.
Clone	A full local copy of a repository, including history.
Commit	A saved snapshot of the project with author, message and parent(s).
Conflict	When Git cannot combine two changes to the same lines automatically.
Detached HEAD	HEAD points directly at a commit instead of a branch.
Diff	The line-by-line difference between two versions.
Fast-forward	A merge that only moves a branch pointer forward, without a merge commit.
Fetch	Download new data from a remote without changing your files.
Fork	Your own server-side copy of someone else's repository.
HEAD	Pointer to your current position, normally the current branch.
Hook	A script Git runs automatically at an event, such as before a commit.
Hunk	One block of changed lines in a diff.
Index	Another name for the staging area.
Issue	A GitHub item tracking a bug, task or idea.
LFS	Large File Storage: keeps big files outside normal Git storage.
Merge	Combine the histories of two branches.
Merge commit	A commit with two or more parents, created by a three-way merge.
Origin	Default name for the remote you cloned from.
ORIG_HEAD	Where HEAD was before a merge, rebase or reset.
Pull	Fetch and then merge (or rebase) into the current branch.
Pull request (PR)	A GitHub proposal to merge a branch, with review and discussion.

Term	Meaning
Push	Upload local commits to a remote.
Rebase	Replay commits on top of a new base commit, creating new commits.
Ref	Any name that points to a commit: a branch, tag or HEAD.
Reflog	Local log of every position HEAD and branches have had.
Remote	A named link to another copy of the repository, usually on a server.
Remote-tracking branch	Your local memory of a remote branch, e.g. `origin/main`.
Repository	A project plus its complete history, stored in `.git`.
Revert	A new commit that undoes an earlier one.
SHA / hash	The unique ID of a Git object, computed from its content.
Squash	Combine several commits into one.
Staging area	Where you prepare the next commit.
Stash	A temporary shelf for uncommitted changes.
Submodule	A repository included inside another at a fixed commit.
Tag	A fixed name for a commit, usually a version.
Tree	Git's stored listing of a folder.
Upstream	The remote branch your branch tracks; also the original repository of a fork.
Working tree	The files on your disk that you edit.
Workflow	In GitHub Actions, an automated process; in teams, an agreed way of using branches.

APPENDIX C

Hands-on Exercises

These labs build on each other. Use a practice folder and a practice GitHub repository. Each lab ends with a **check**: a command whose output proves you succeeded.

Lab 1. First commits

1. Configure your name, email and default branch.
2. Create a folder `recipes`, run `git init`.
3. Create `README.md` and commit it. Create `pasta.md` and commit it separately.
4. Edit `pasta.md`, view the change with `git diff`, then commit.

Check: `git log --oneline` shows three commits with clear messages.

Lab 2. Ignoring files

1. Create `notes.tmp`, a folder `build/` with a file inside, and `secret.env`.
2. Write a `.gitignore` that ignores all three. Commit the `.gitignore`.
3. Commit `secret.env` on purpose, then stop tracking it while keeping the file.

Check: `git status` is clean and `git ls-files` does not list `secret.env`.

Lab 3. Exploring history

1. Make at least six commits that edit `pasta.md` and `README.md`.
2. Use `git log -S` to find the commit that first added the word "garlic".
3. Show `pasta.md` as it was three commits ago, without changing your files.

Check: you can name the hash of the "garlic" commit and print the old file with `git show`.

Lab 4. Undo everything

1. Edit a file and discard the edit with `git restore`.
2. Stage a file and unstage it.
3. Make a commit with a typo in the message and fix it with `--amend`.
4. Make two commits and undo them with `git reset --soft HEAD~2`, then recommit as one.
5. Run `git reset --hard HEAD~1`, then recover the commit with `git reflog`.

Check: `git log` contains the recovered commit.

Lab 5. Branching and a planned conflict

1. Create branch `spicy`, change one line of `pasta.md`, commit.
2. Switch to `main`, change the same line differently, commit.
3. Merge `spicy` into `main`, resolve the conflict, finish the merge.

Check: `git log --oneline --graph` shows the two branches joining in a merge commit.

Lab 6. GitHub and remotes

1. Set up SSH (or `gh auth login`) and test it.
2. Create an empty GitHub repository and push `recipes` to it with `-u`.
3. Edit `README.md` on the GitHub website and commit there.
4. Run `git status` locally (it does not know yet), then `git fetch`, then `git status` again, then `git pull`.

Check: local and GitHub histories match; `git status` says "up to date".

Lab 7. Pull requests in pairs

1. Add your partner as a collaborator.
2. Each person creates a branch, adds a new recipe file and opens a PR.
3. Review each other's PR: leave one line comment and one suggestion.
4. Address the review, then merge with *Squash and merge*. Delete the branches.

Check: `main` on GitHub has one squashed commit per PR.

Lab 8. Clean history with interactive rebase

1. On a new branch, make five small commits, including two "fix typo" commits.
2. Use `git rebase -i` to combine the typo fixes into the commits they fix, and reword one message.
3. Push the branch, rebase it once more, and push with `--force-with-lease`.

Check: `git log --oneline main..HEAD` shows three tidy commits.

Lab 9. Automation

1. Add a small Python function with a `pytest` test.
2. Add the CI workflow from Chapter 14.
3. Open a PR that breaks the test, watch CI fail, fix it, watch it pass.
4. Protect `main` so that CI must pass before merging.

Check: the PR shows a green check and the README shows a passing badge.

Lab 10. Release and publish

1. Tag the current `main` as `v1.0.0` (annotated) and push the tag.
2. Create a GitHub release with generated notes.
3. Enable GitHub Pages from the `main` branch and add an `index.html` that links to your recipes.
4. Add a `CITATION.cff` file.

Check: the release page, the live website and the *Cite this repository* button all work.

Challenge. Rescue the repository

Ask a friend to break a copy of your repository in three ways (for example: reset away commits, delete a branch, leave a half-finished rebase). Using only `status`, `log`, `reflog` and the commands in Chapter 20, restore everything.

APPENDIX D

Further Resources

Official documentation

- *Pro Git* book, free online: `git-scm.com/book`
- Git reference manual: `git-scm.com/docs`
- GitHub Docs: `docs.github.com`
- GitHub CLI manual: `cli.github.com/manual`

Interactive learning

- *Learn Git Branching*, a visual game for branching and rebasing: `learngitbranching.js.org`
- *GitHub Skills*, hands-on courses inside real repositories: `skills.github.com`

Quick help when things go wrong

- *Oh Shit, Git!?!*, plain-language fixes for common mistakes: `ohshitgit.com`
- Git's own built-in help: `git help <command>` and `git help -g`

Conventions and standards

- Conventional Commits: `conventionalcommits.org`
- Semantic Versioning: `semver.org`
- Choosing a license: `choosealicense.com`
- Gitignore templates: `github.com/github/gitignore`
- Citation File Format: `citation-file-format.github.io`

Research software and reproducibility

- *The Turing Way*, a community handbook on reproducible research: `book.the-turing-way.org`
- Zenodo and GitHub integration: `zenodo.org`
- Software Carpentry lesson "Version Control with Git": `swcarpentry.github.io/git-novice`

Final advice

Git rewards practice more than memorisation. Use it for everything: code, papers, slides, notes, even this kind of guide. Commit small and often, write messages for your future self, branch freely, and remember that with the reflog, almost every mistake can be undone.